



三维点云处理技术 和深度学习在点云处理中的应用

三维点云表征概述

索传哲



内容概要

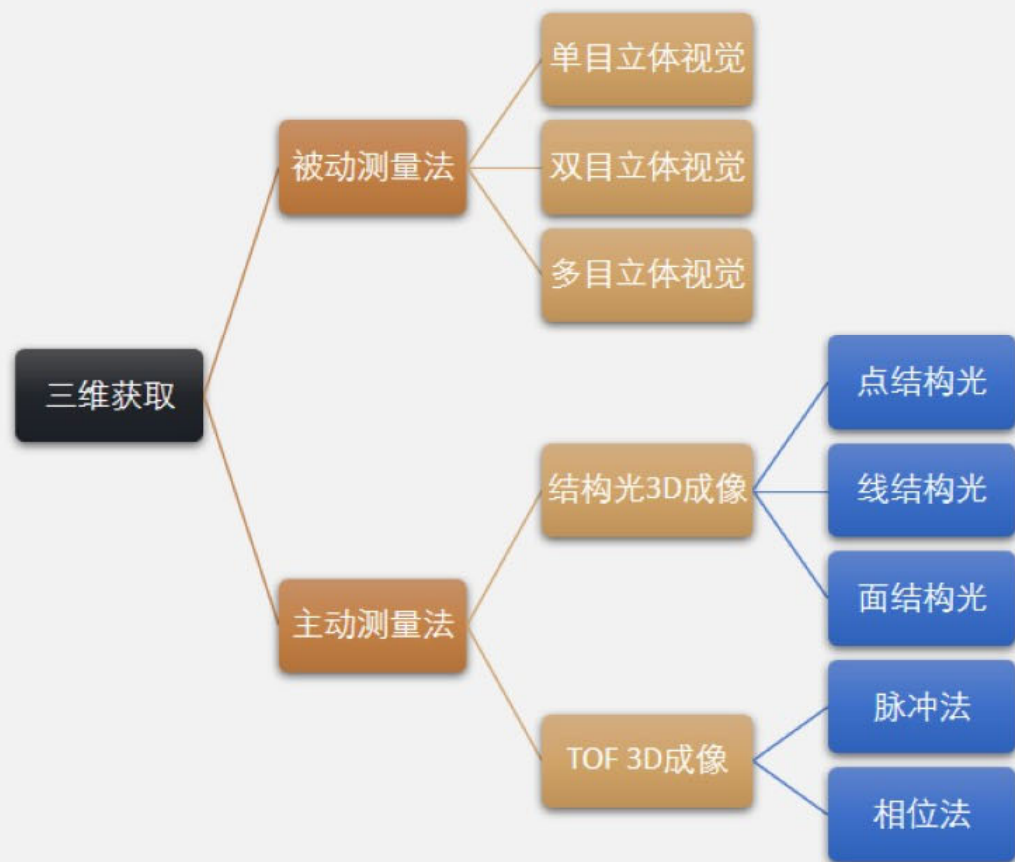
三维数据的获取方式及原理

三维信息的表征形式

点云的基本特征和描述

三维数据的获取方式及原理

➤ 三维数据的获取方式:



三维数据的获取方式及原理

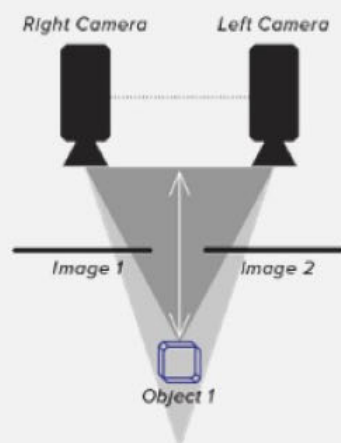
➤ 三维数据的获取方式:

立体视觉测量法

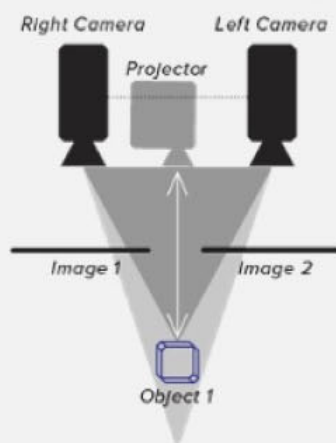
结构光3D成像法

飞行时间TOF法

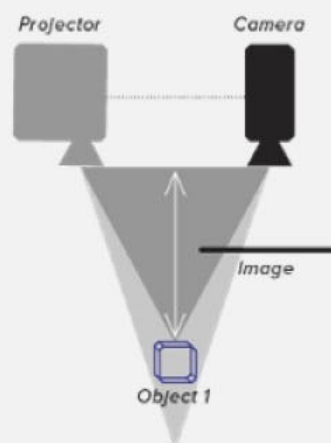
PASSIVE STEREO



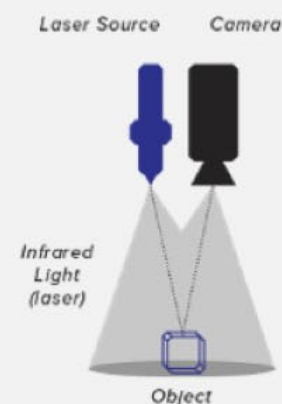
ACTIVE STEREO



STRUCTURED LIGHT



TIME OF FLIGHT



<https://medium.com/@DAQRI/depth-cameras-for-mobile-ar-from-iphones-to-wearables-and-beyond-ea29758ec280>

三维数据的获取方式及原理

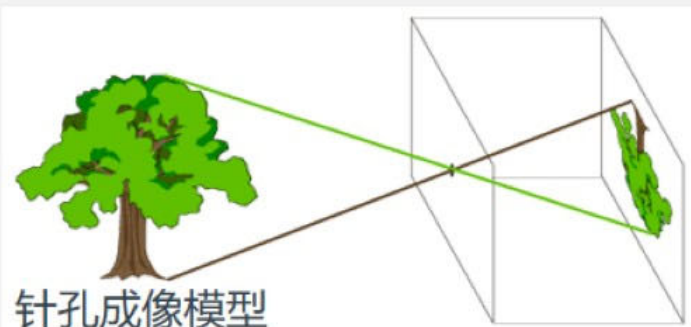
➤ 立体视觉测量法:

相机成像模型 ☒

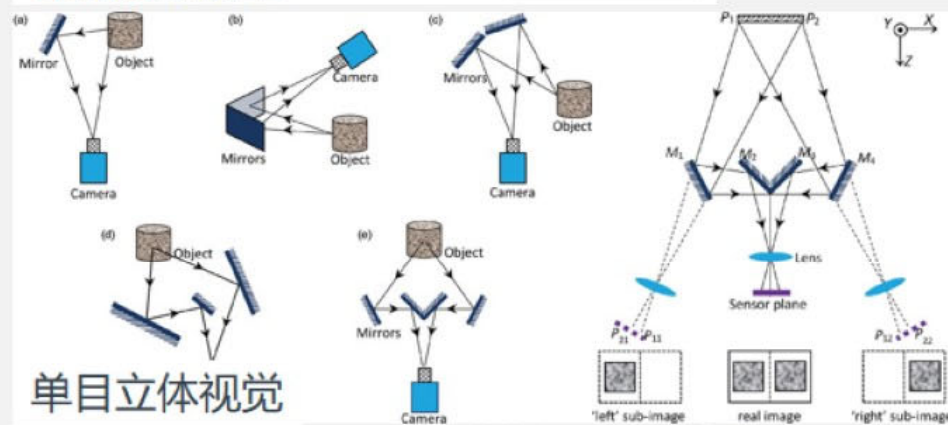
单目立体视觉

双目立体视觉 ☒

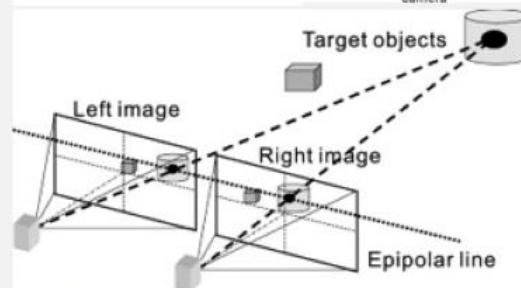
多目立体视觉



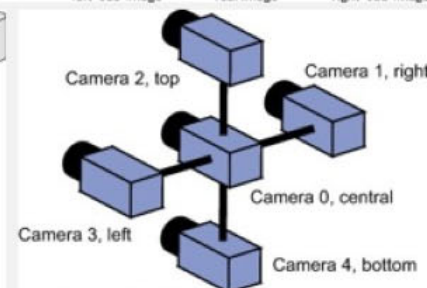
针孔成像模型



单目立体视觉



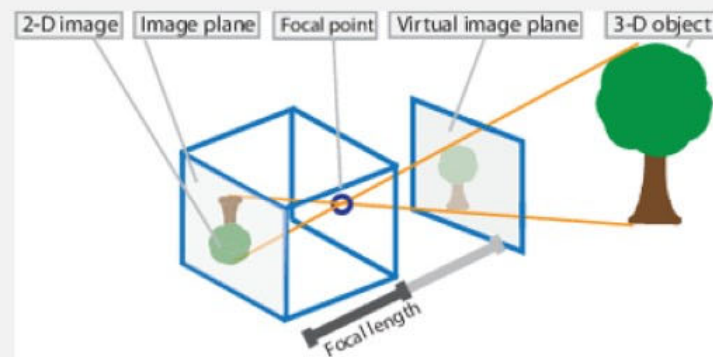
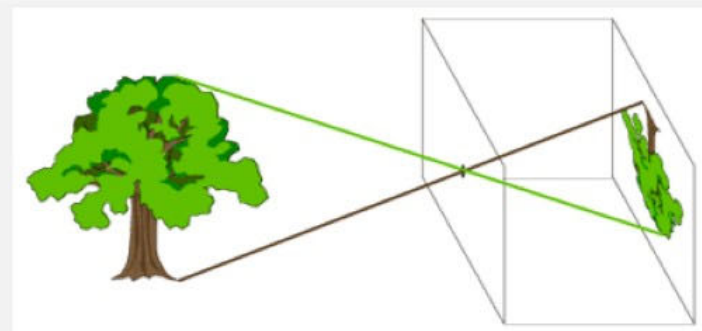
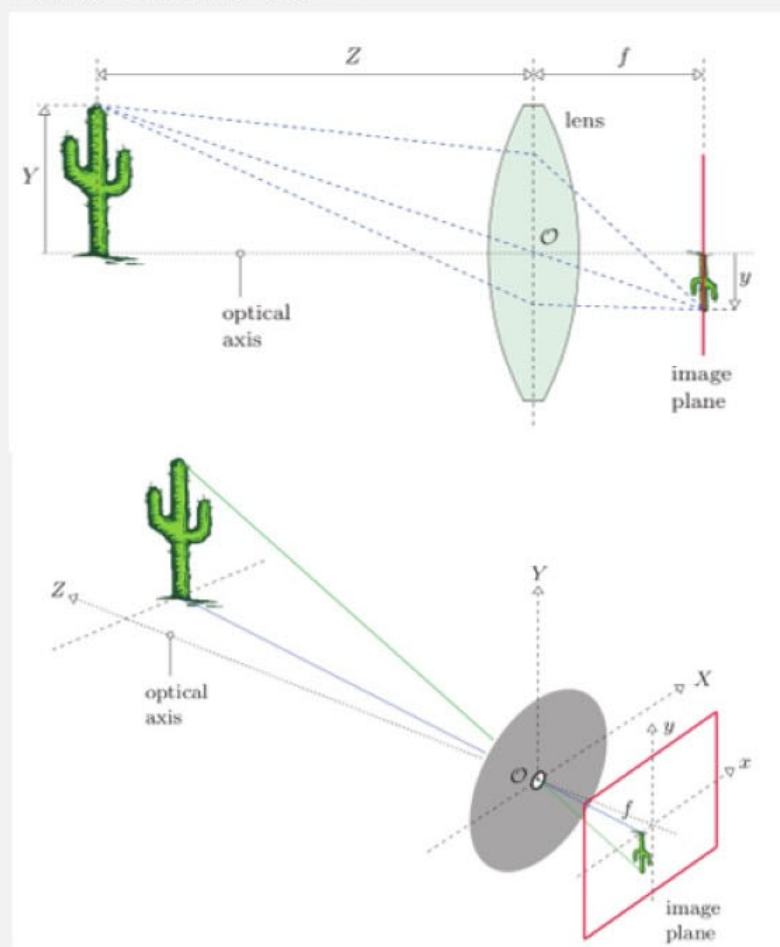
双目立体视觉



多目立体视觉

三维数据的获取方式及原理

➤ 相机成像模型:



针孔成像模型

- 凸透镜成像原理: 光心、光轴、焦距、焦平面

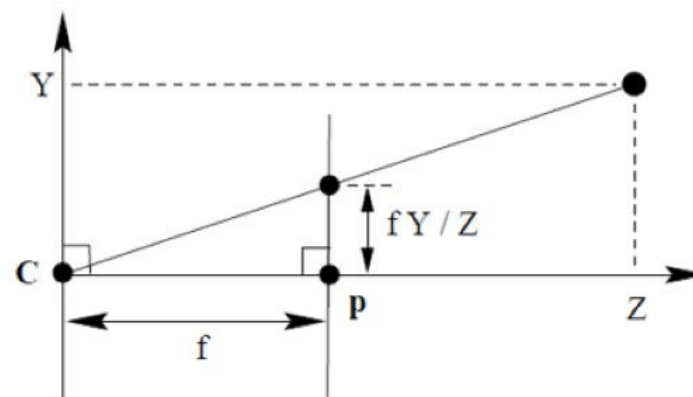
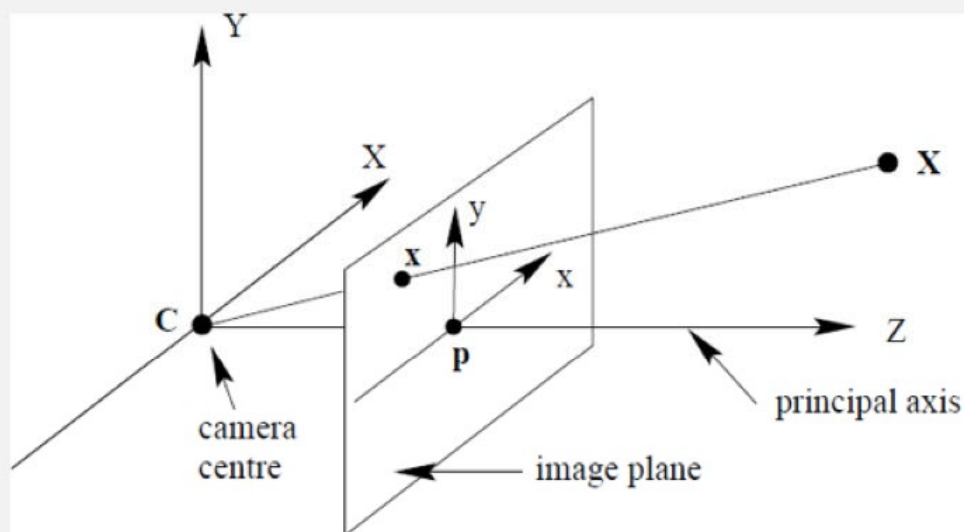
$$\frac{1}{u} + \frac{1}{v} = \frac{1}{f}$$

- 简化小孔成像: 相似三角形计算

$$\frac{x}{f} = \frac{X}{Z} \quad \frac{y}{f} = \frac{Y}{Z}$$

三维数据的获取方式及原理

➤ 相机成像模型:



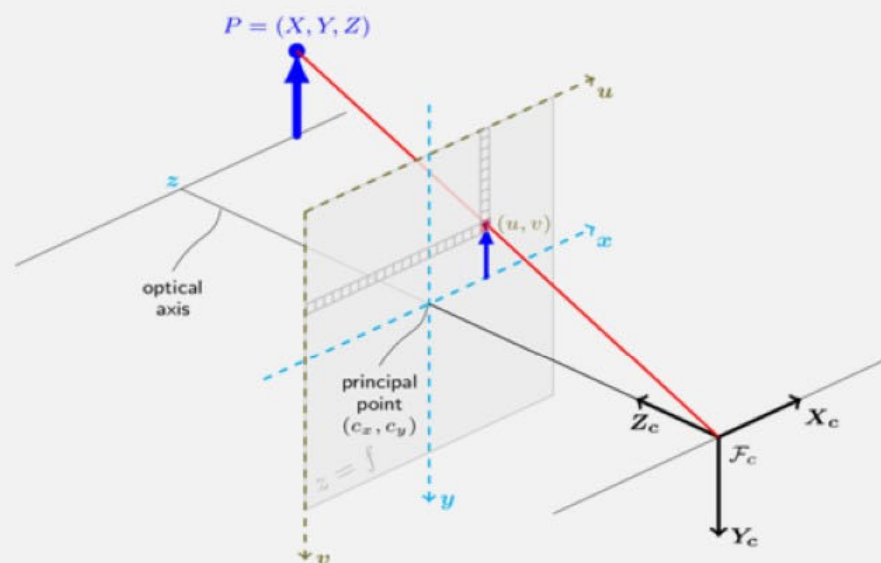
$$\begin{cases} \frac{x}{f} = \frac{X}{Z} \\ \frac{y}{f} = \frac{Y}{Z} \end{cases} \Rightarrow \begin{cases} X = x \frac{Z}{f} \\ Y = y \frac{Z}{f} \end{cases} \quad \begin{cases} x = \frac{fX}{Z} \\ y = \frac{fY}{Z} \end{cases} \Rightarrow \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} fX \\ fY \\ Z \end{bmatrix} = \begin{bmatrix} f & & \\ & f & \\ & & 1 \end{bmatrix} \begin{bmatrix} 1 & & & 0 \\ & 1 & & 0 \\ & & 1 & 0 \\ & & & 1 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

K $[R|t]$

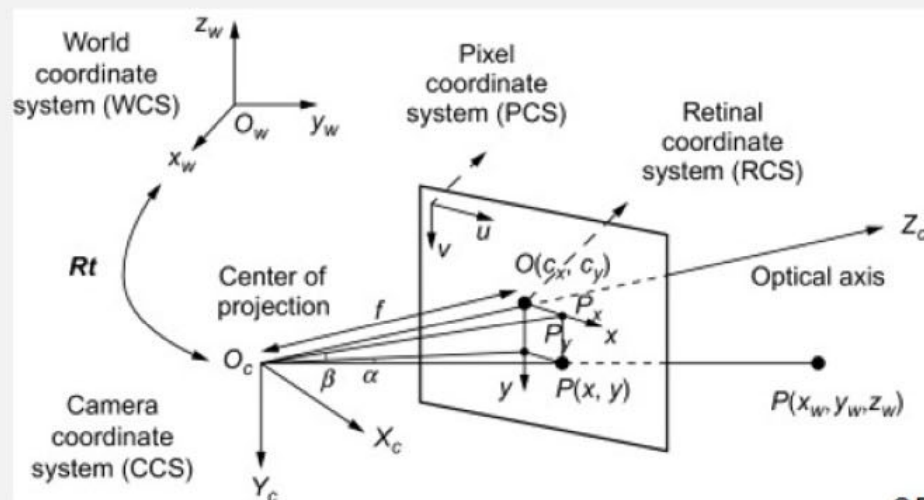
Richard Hartley and Andrew Zisserman. "Multiple View Geometry in Computer Vision". Cambridge University Press, Second Edition 2004.

三维数据的获取方式及原理

➤ 相机成像模型:



$$\begin{cases} x = s_x(u - c_x) \\ y = s_y(v - c_y) \end{cases} \Rightarrow \begin{cases} X = (u - c_x) \frac{Z}{f_x} \\ Y = (v - c_y) \frac{Z}{f_y} \end{cases} \quad \begin{cases} X = x \frac{Z}{f} \\ Y = y \frac{Z}{f} \end{cases} \quad \begin{cases} f_x = \frac{f}{s_x} & f_y = \frac{f}{s_y} \end{cases}$$



像素坐标
(齐次坐标)

$$Z \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \Rightarrow \begin{cases} u = \frac{x}{s_x} + c_x \\ v = \frac{y}{s_y} + c_y \end{cases}$$

内参矩阵

外参矩阵

3D物理
坐标

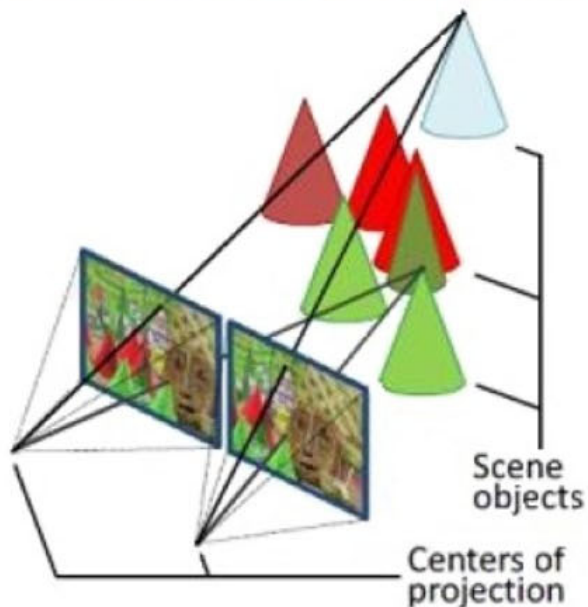
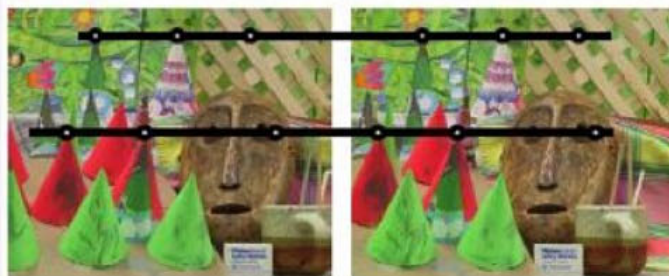
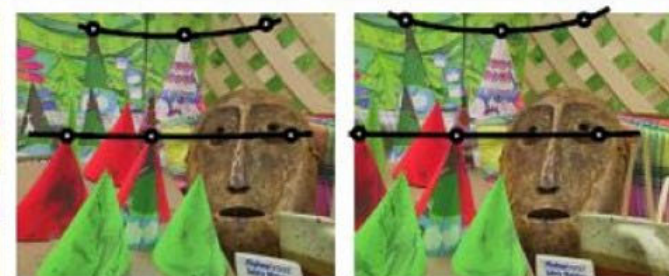
$$\begin{bmatrix} f_x X + c_x Z \\ f_y Y + c_y Z \\ Z \end{bmatrix} = \begin{bmatrix} f_x & c_x \\ & f_y & c_y \\ & & 1 \end{bmatrix} \begin{bmatrix} 1 & & & 0 \\ & 1 & & 0 \\ & & 1 & 0 \\ & & & 1 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

$K \quad [R|t]$

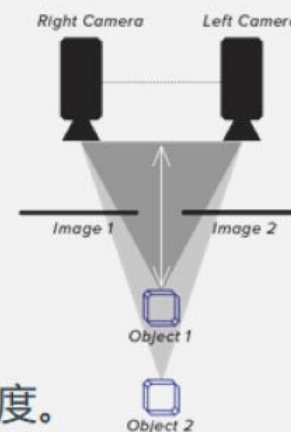
<https://www.sciencedirect.com/topics/engineering/pinhole-camera-model>

三维数据的获取方式及原理

➤ 双目立体视觉:



PASSIVE STEREO

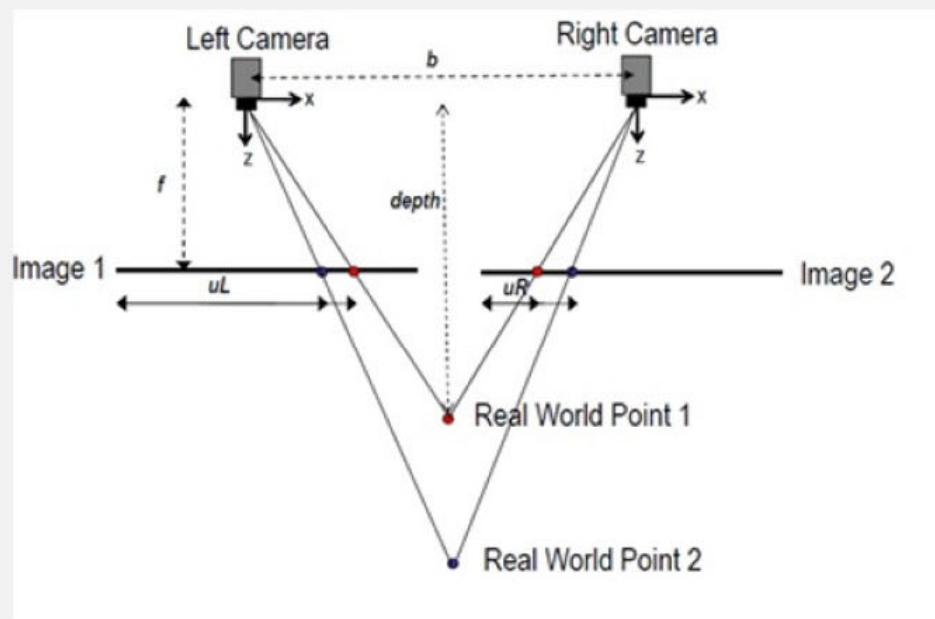


双目视觉是利用视差原理计算深度。两幅图片因为相机视角不同带来的图像的差别构成视差

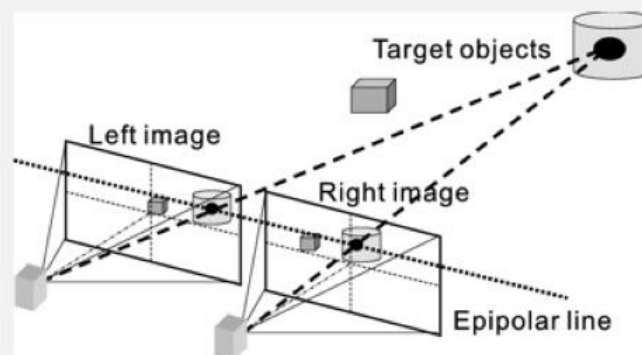
- 1、首先需要对双目相机进行标定,得到两个相机的内外参数、单应矩阵。
- 2、根据标定结果对原始图像校正,校正后的两张图像位于同一平面且互相平行。
- 3、对校正后的两张图像根据极线约束进行像素点匹配。
- 4、根据匹配结果计算每个像素的深度,从而获得深度图

三维数据的获取方式及原理

➤ 双目立体视觉:



理想情况：左右相机内参相同且像平面共面



双目立体视觉

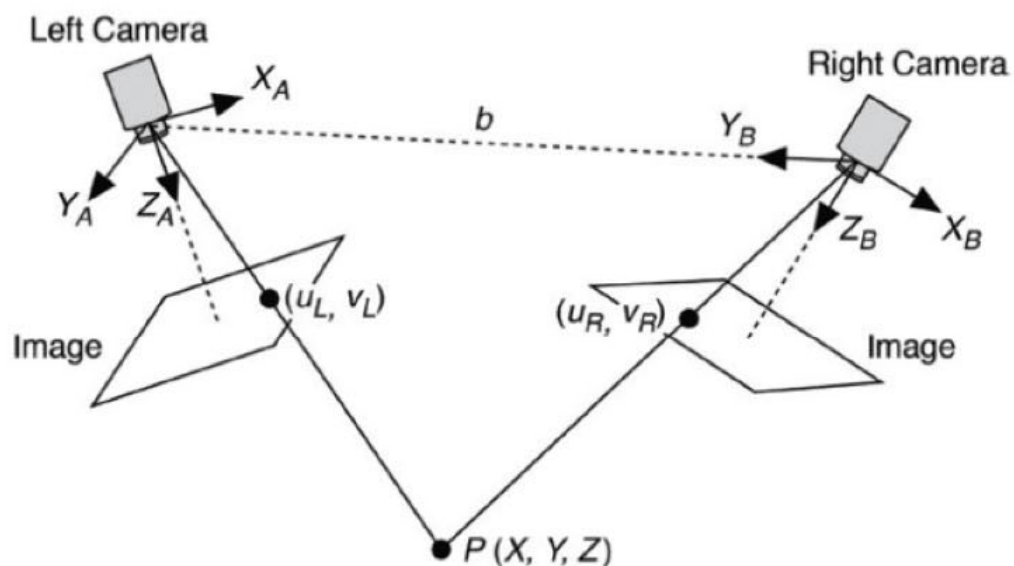
$$\frac{\text{depth} - f}{b - (uL - uR)} = \frac{\text{depth}}{b}$$

⇓

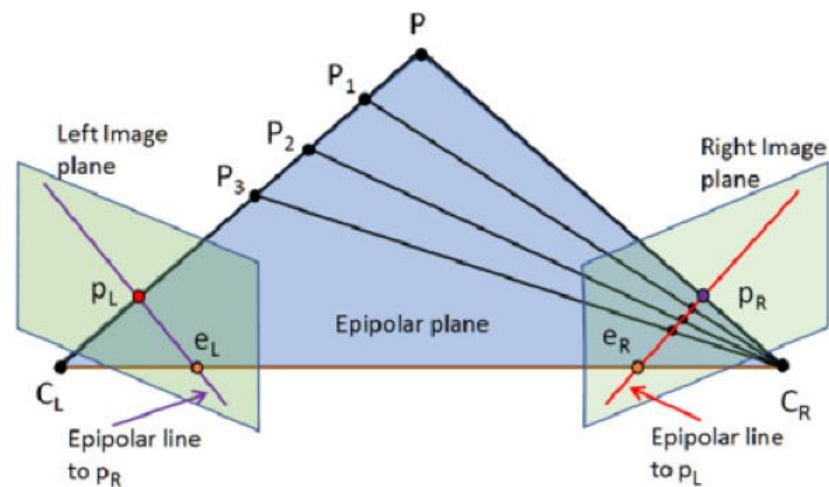
$$\text{depth} = \frac{bf}{\Delta u} \quad \Delta u = uL - uR \text{ 视差}$$

三维数据的获取方式及原理

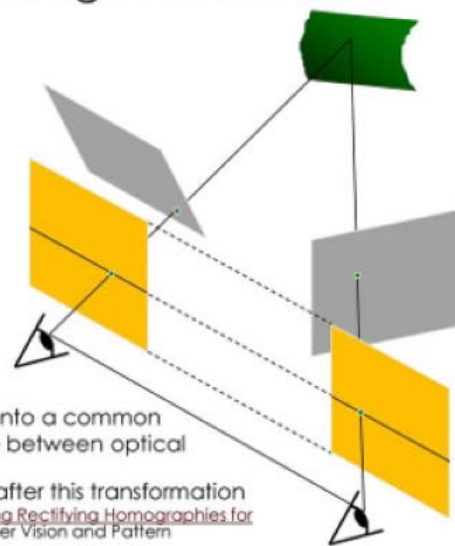
➤ 双目立体视觉:



非理想情况：左右相机像平面不共面，甚至相机内参不同
图像校正+极线约束



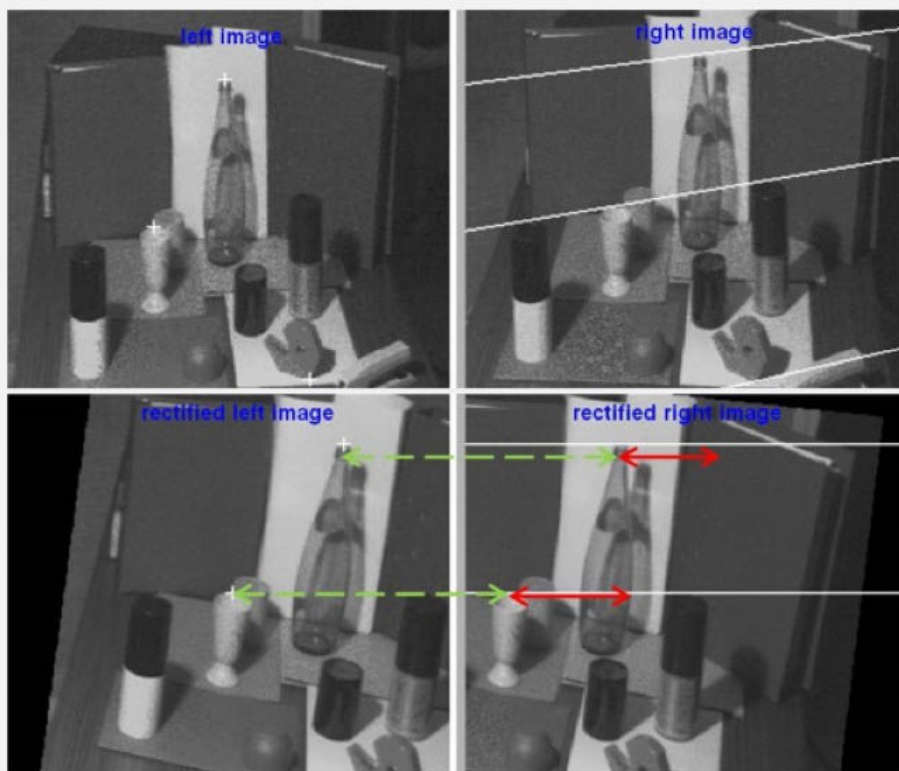
Stereo image rectification



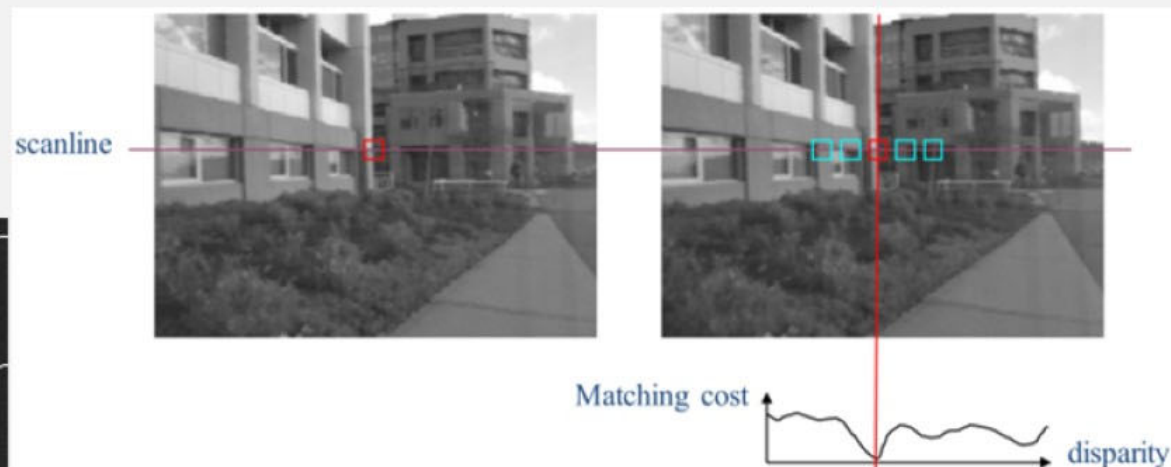
- reproject image planes onto a common plane parallel to the line between optical centers
- pixel motion is horizontal after this transformation
- C. Loop and Z. Zhang. Computing Rectifying Homographies for Stereo Vision. IEEE Conf. Computer Vision and Pattern Recognition, 1999.

三维数据的获取方式及原理

➤ 双目立体视觉:



图像校正+极线约束



滑动搜索框

左图 x, y 位置像素和右图 x, y, d 位置像素
的差异计算公式

像素差绝对值和 (SAD) :

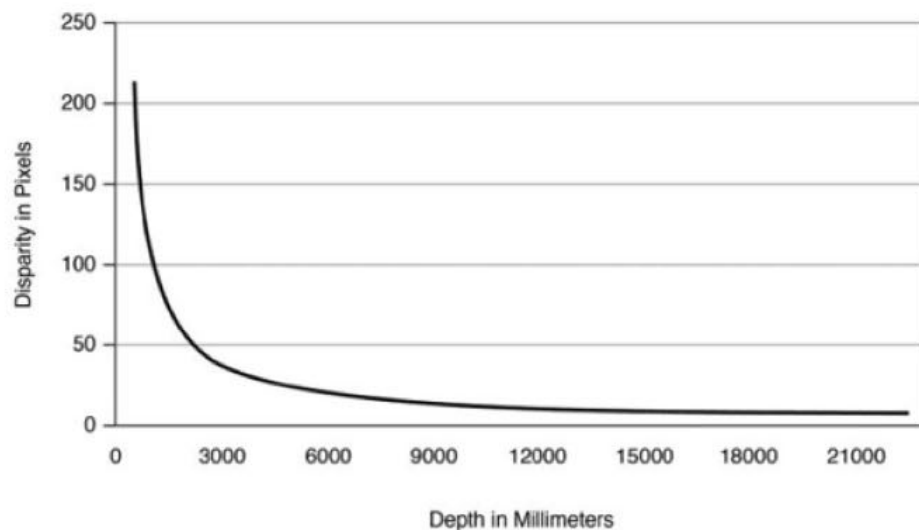
$$f(x, y, d) = \sum_{d_x=-W}^W \sum_{d_y=-W}^W |I_L(x + d_x, y + d_y) - I_R(x - d + d_x, y + d_y)|$$

像素差平方和 (SSD) :

$$f(x, y, d) = \sum_{d_x=-W}^W \sum_{d_y=-W}^W (I_L(x + d_x, y + d_y) - I_R(x - d + d_x, y + d_y))^2$$

三维数据的获取方式及原理

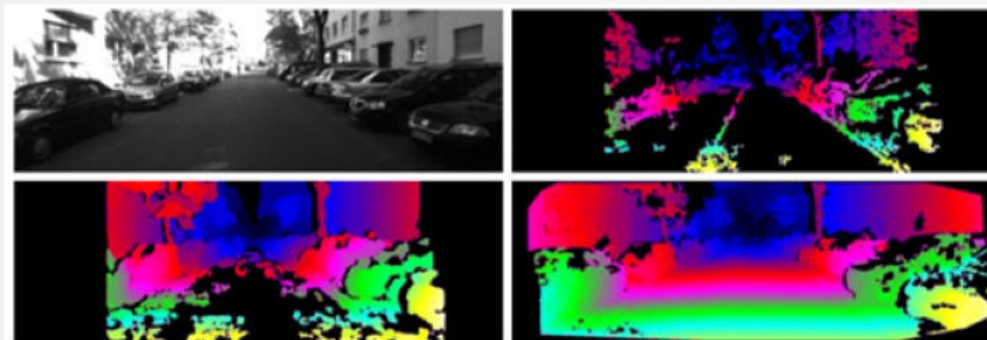
➤ 双目立体视觉:



视差随深度变化曲线
($f=8\text{mm}$, $b=10\text{cm}$, $\text{pixel size}=7.5\mu\text{m}$)

评价算法好坏的几个方面:

对边界处深度的估计, 无纹理地方的估计, 渐变面的深度估计, 遮掩地方的估计, 还有计算的时间, 内存



对边缘不友好, 滑动窗口大小对效果很有影响



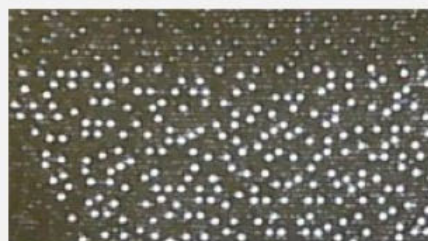
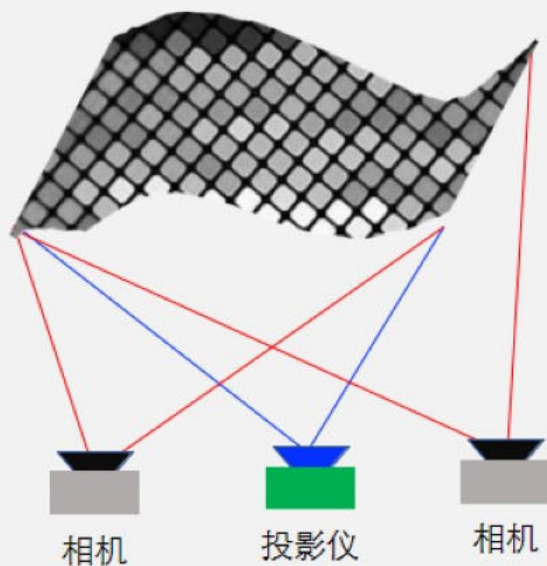
不适用于单调缺乏纹理的场景。因为双目视觉根据视觉特征进行图像匹配, 所以对于缺乏视觉特征的场景(如天空、白墙、沙漠等)会出现匹配困难甚至匹配失败

<https://www.techbriefs.com/component/content/article/tb/features/articles/14925?start=1>



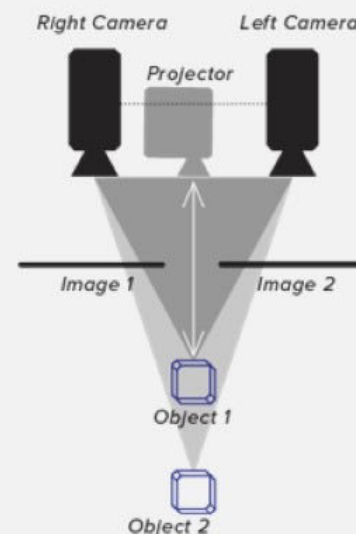
三维数据的获取方式及原理

➤ 双目立体视觉:



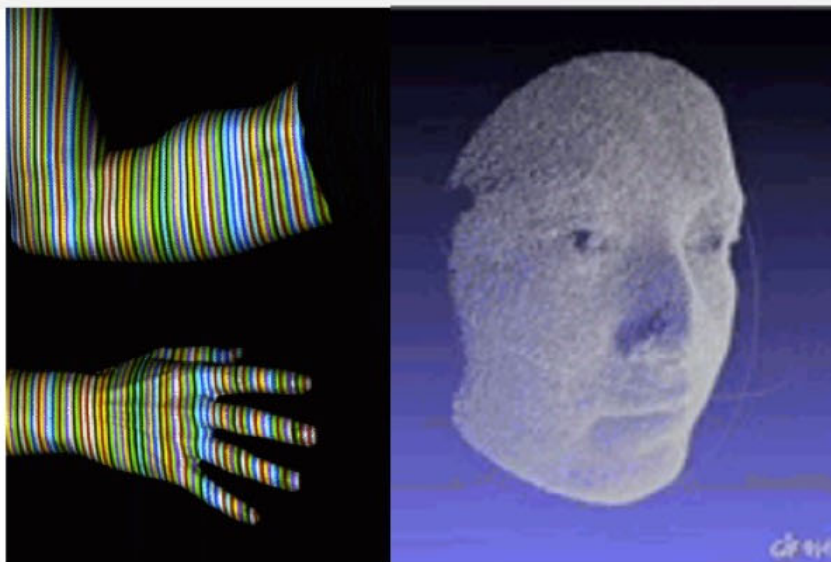
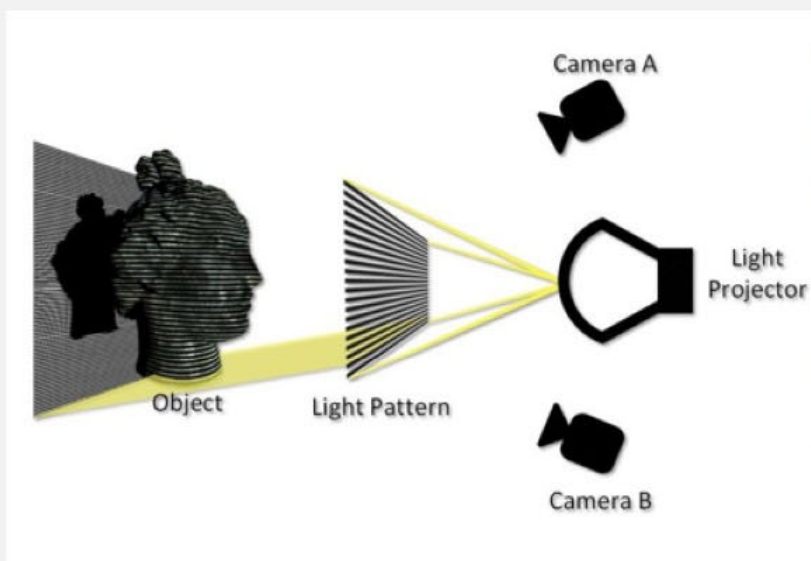
- 利用投影在物体上的图案，帮助两个相机计算双目匹配
- 有多种可能的投影图案，一个方案是使用伪随机散斑

ACTIVE STEREO



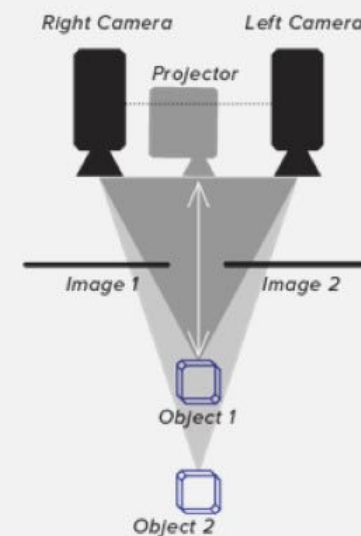
三维数据的获取方式及原理

➤ 双目立体视觉:



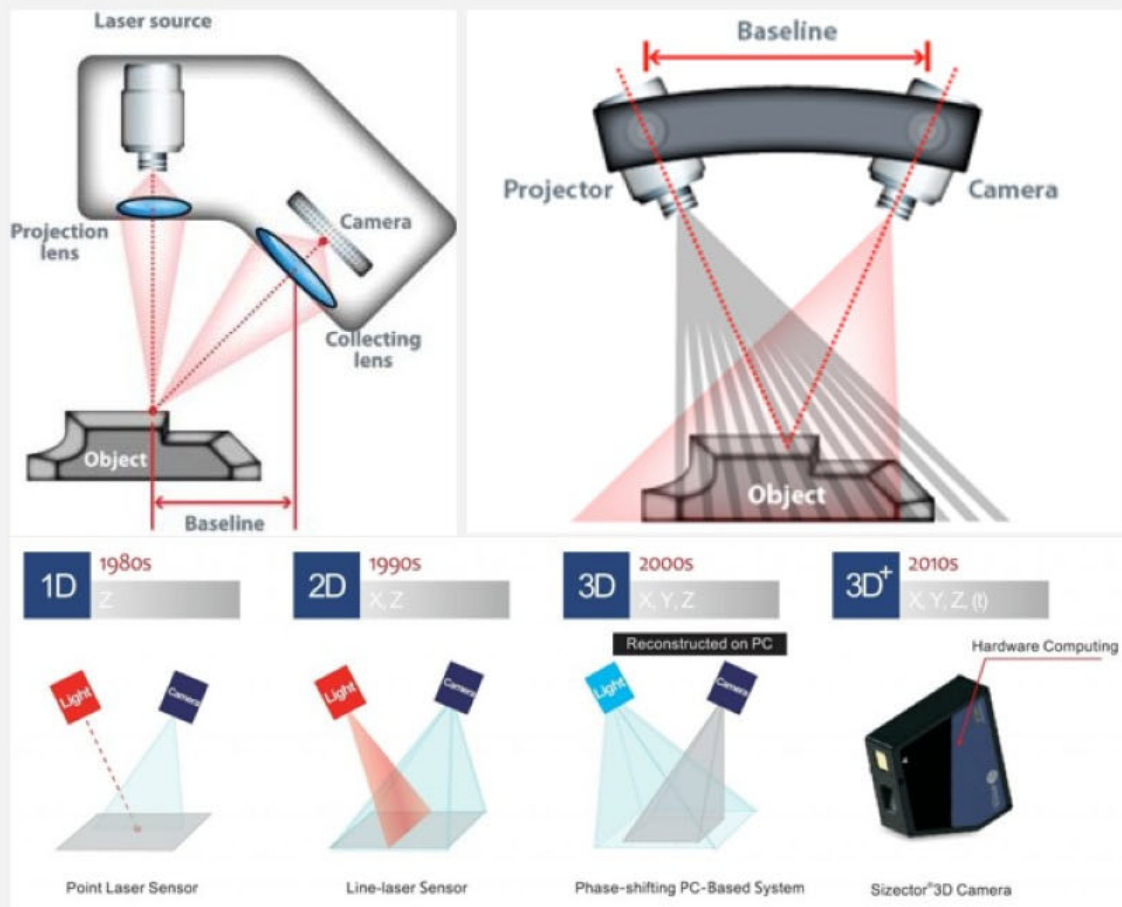
- 利用投影在物体上的图案，帮助两个相机计算双目匹配
- 有多种可能的投影图案，一个方案是使用伪随机散斑

ACTIVE STEREO

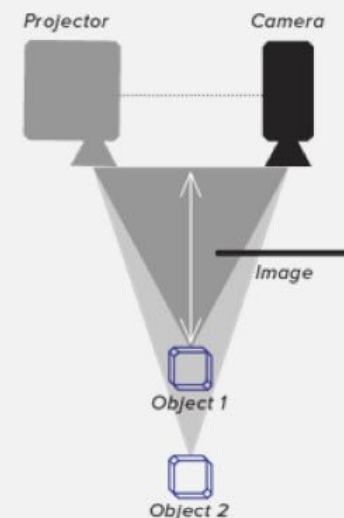


三维数据的获取方式及原理

➤ 结构光3D成像法:



STRUCTURED LIGHT



• 点结构光

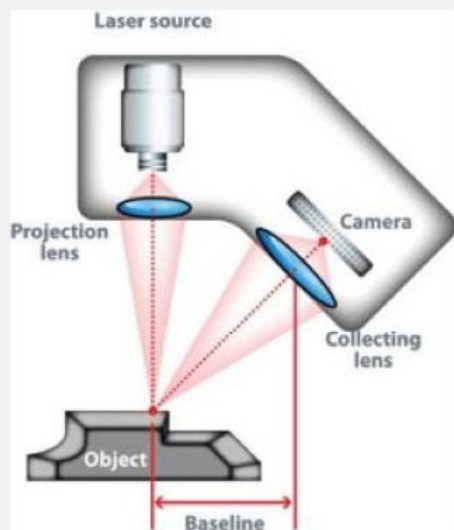
使用单个点结构光 (e.g. 激光点) 扫描整个对象。传感器检测物体反射的结构光, 使用三角测量法, 计算从物体到扫描仪的距离。

• 线结构光

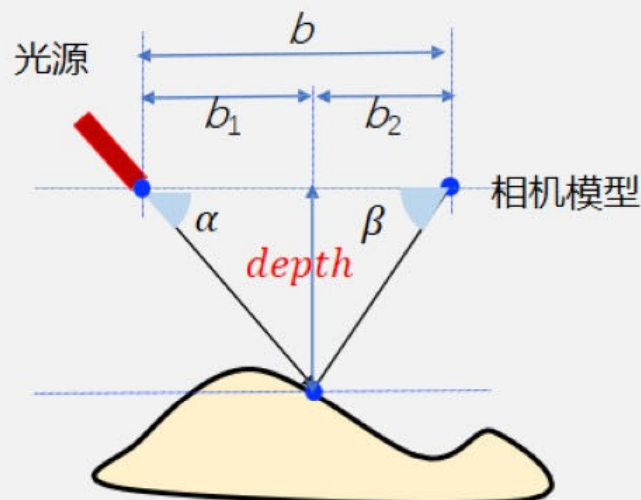
使用三角测量法, 利用投影仪在物体投射一系列结构化线束。通过检测光线的边缘, 计算出从扫描仪到物体表面的距离。

三维数据的获取方式及原理

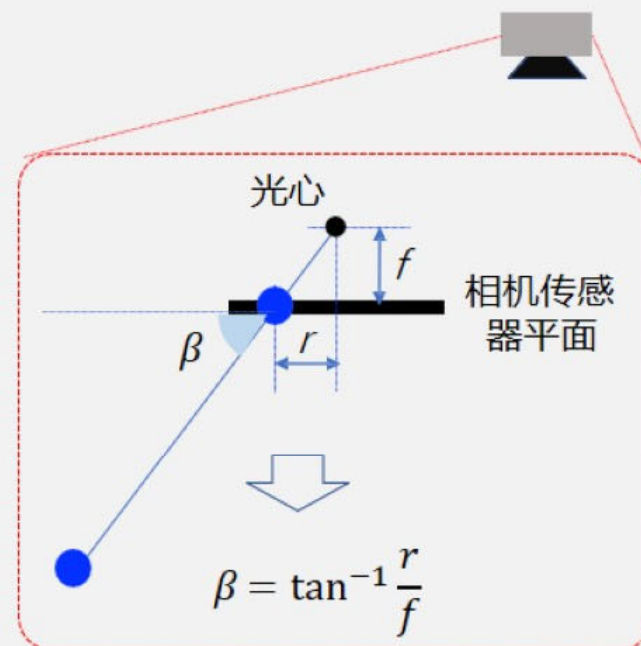
➤ 结构光3D成像法:



• 点结构光



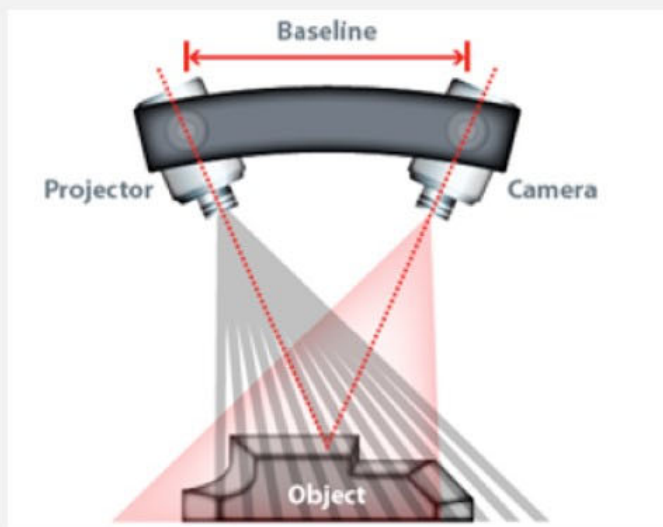
$$\frac{\text{depth}}{\tan \alpha} + \frac{\text{depth}}{\tan \beta} = b$$
$$\Downarrow$$
$$\text{depth} = b \frac{\sin \alpha \sin \beta}{\sin(\alpha + \beta)}$$



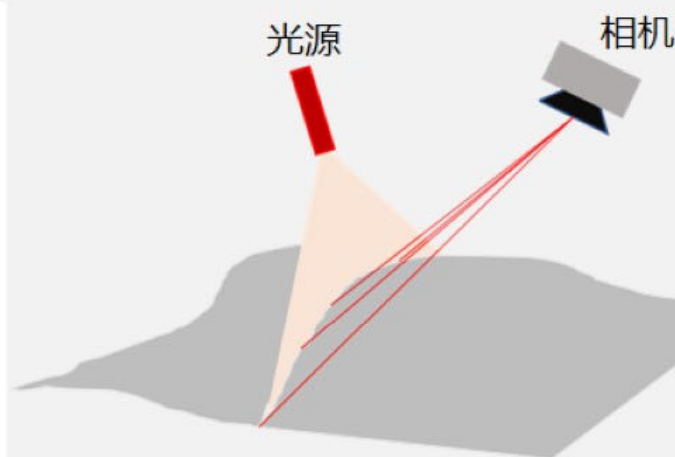
- 从图像中光点位置x计算 β
- 通过光源转动进行光点扫描就能够计算出物体表面各个位置的距离值 **depth**

三维数据的获取方式及原理

➤ 结构光3D成像法:



• 线结构光



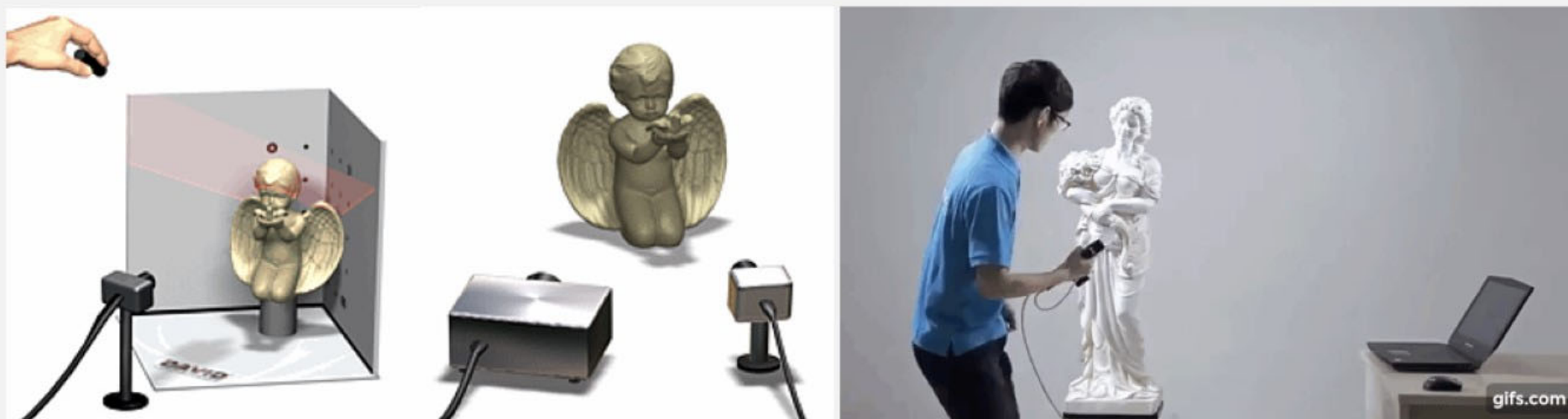
- (x, y) 是相机传感器平面上的点(已知)
- (X, Y, Z) 是对应的空间点的坐标(未知)
- f 是相机内参

- 光源平面在相机坐标系中的平面方程
- 通过针孔相机成像模型求解 (X, Y, Z)
- 转动光源进行线扫描获取物体表面各个位置的坐标

$$\begin{cases} AX + BY + CZ + D = 0 \\ X = x \frac{Z}{f} \\ Y = y \frac{Z}{f} \end{cases} \Rightarrow \begin{cases} Z = -\frac{Df}{Ax + By + Cf} \\ X = -\frac{Dx}{Ax + By + Cf} \\ Y = -\frac{Dy}{Ax + By + Cf} \end{cases}$$

■ 三维数据的获取方式及原理

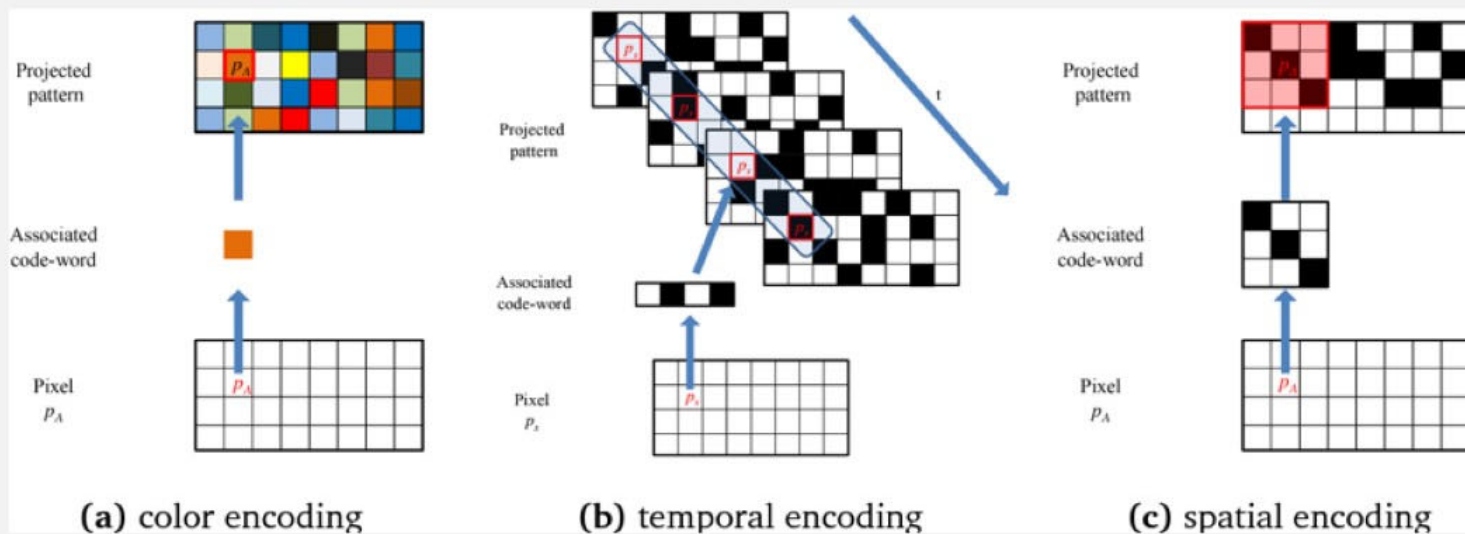
➤ 结构光3D成像法:



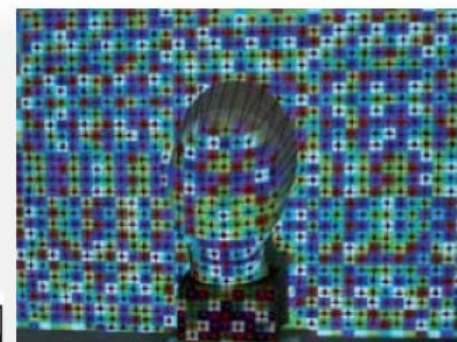
- 投影面结构光到物体表面，需要对面结构光进行编码，知道每个面结构光中的点与所成像中像素的对应关系，然后进行计算
- 直接编码，时分复用，空分复用

三维数据的获取方式及原理

➤ 结构光3D成像法:



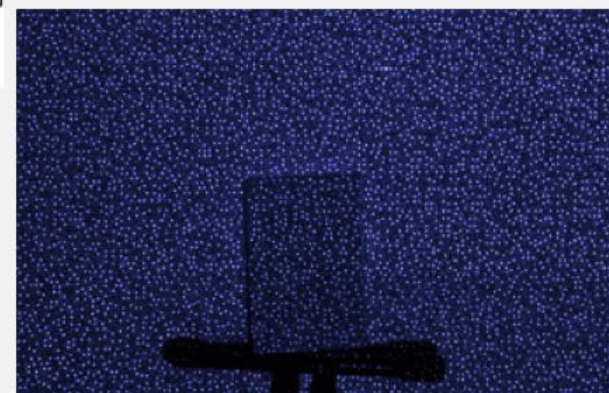
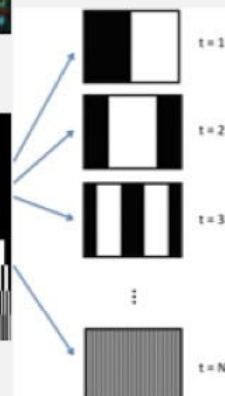
- 直接编码, 时分复用, 空分复用



直接颜色编码



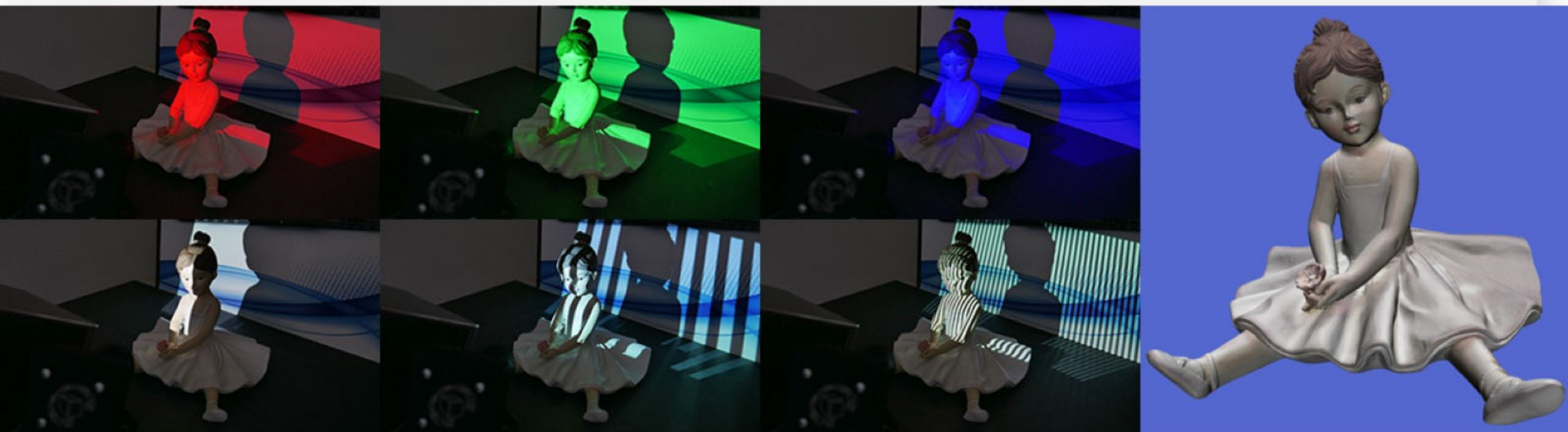
格雷码时分复用编码



空分复用编码

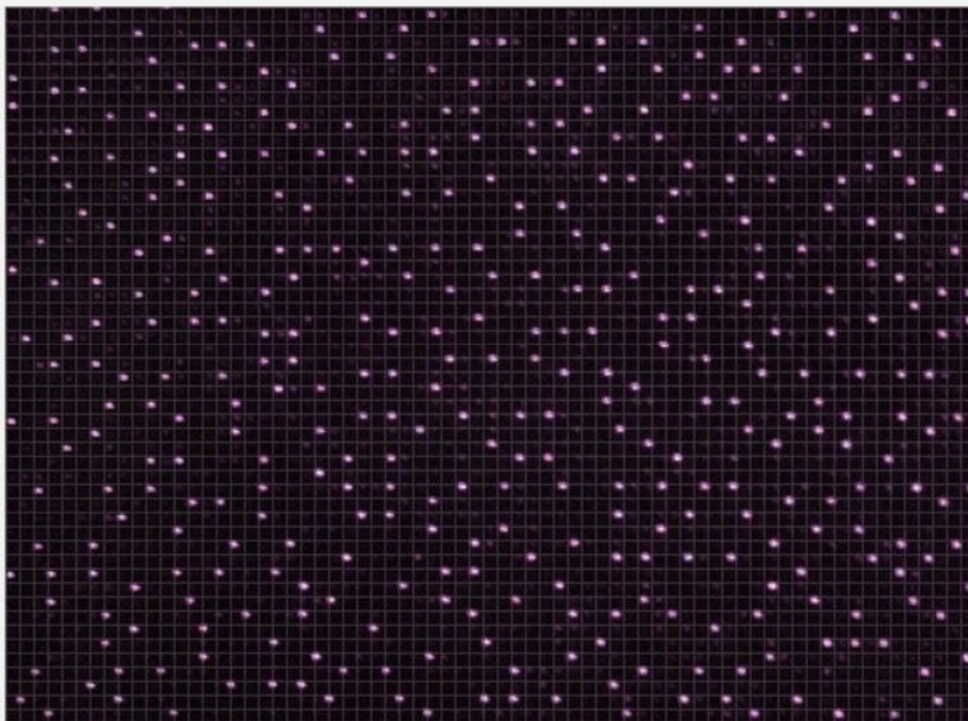
三维数据的获取方式及原理

➤ 结构光3D成像法:

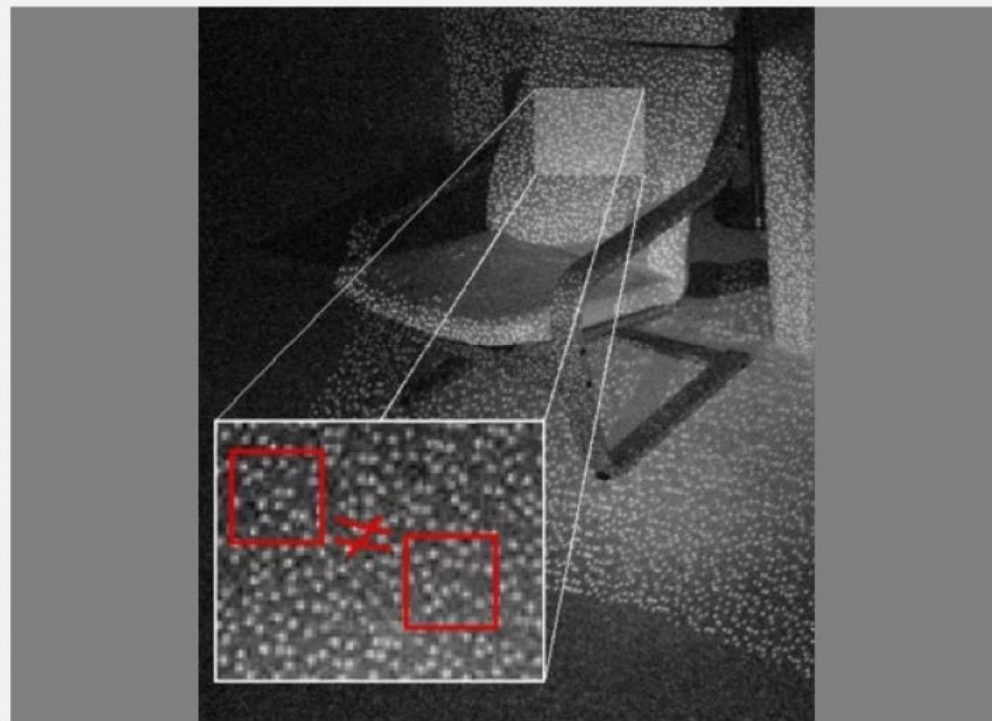


■ 三维数据的获取方式及原理

➤ 结构光3D成像法:



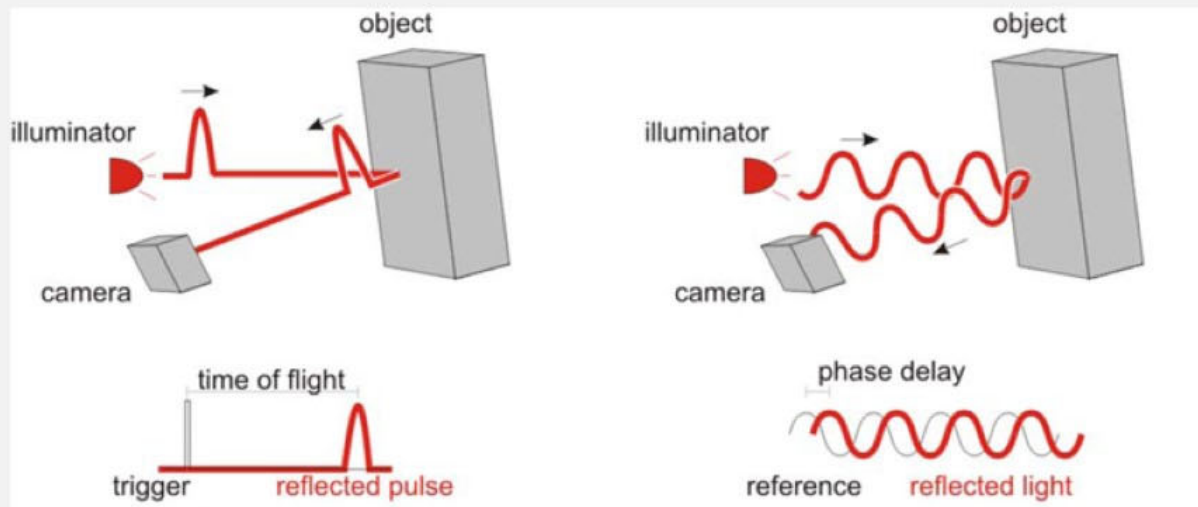
Bitmap from Images





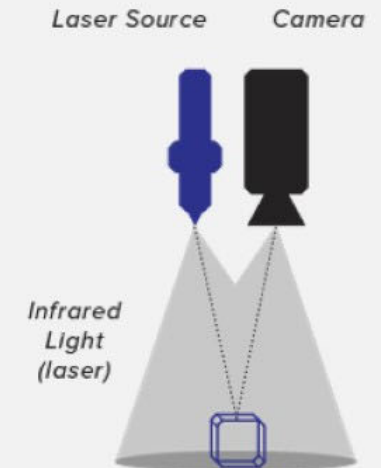
三维数据的获取方式及原理

➤ 飞行时间TOF法:



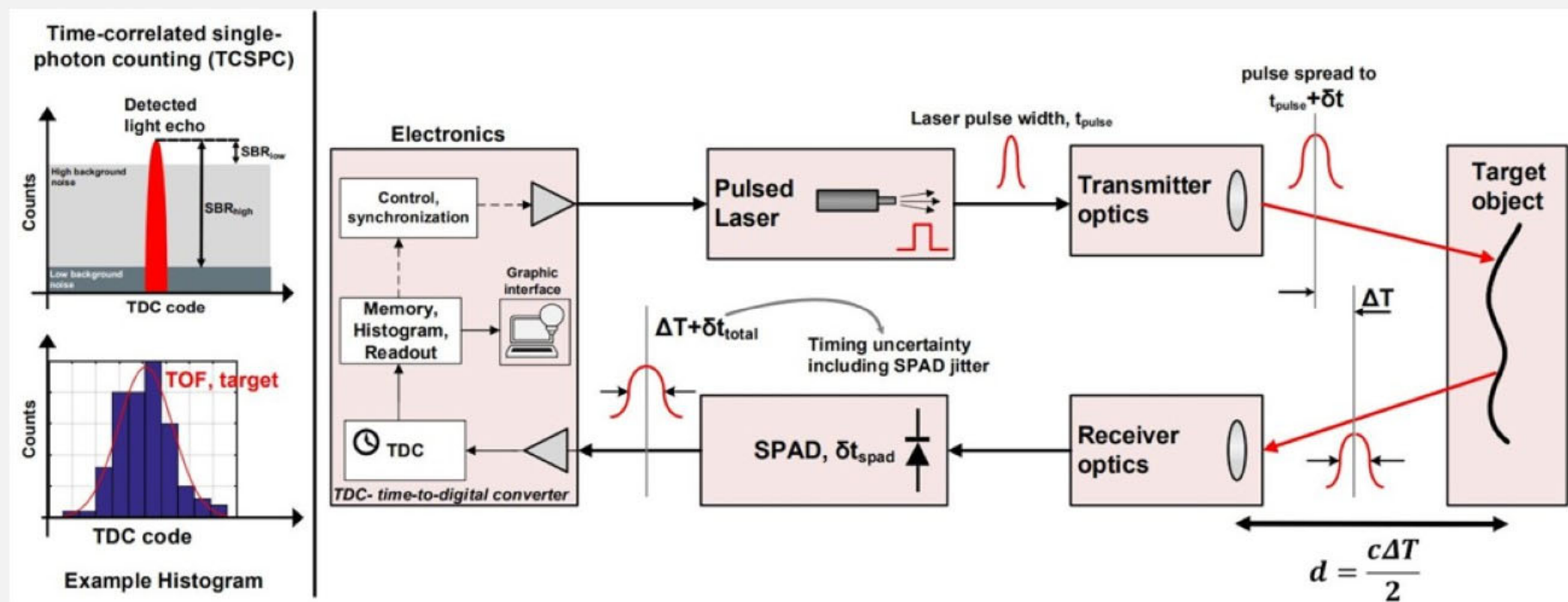
- **脉冲法**
距离的时域检测，通过估计光在被物体反射后从照明器传播到相机所花费的时间来计算。
- **相位法**
距离的频域检测，根据发射光与反射光的相位之间的相位延迟来计算飞行时间。

TIME OF FLIGHT



三维数据的获取方式及原理

➤ 飞行时间TOF法:



理想接收脉冲



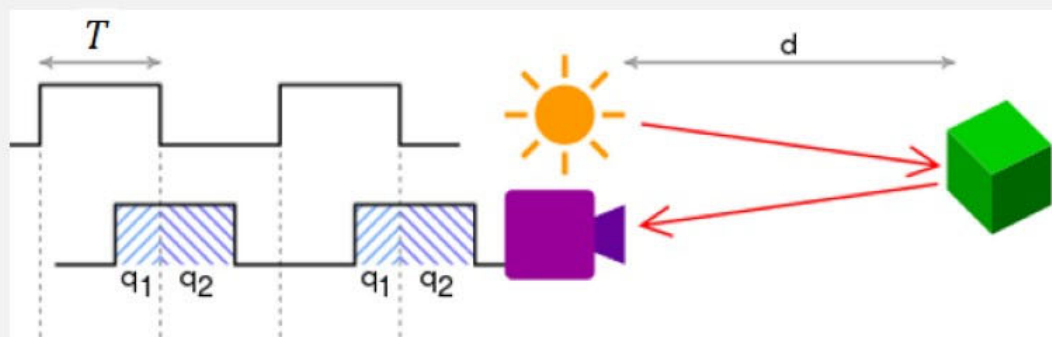
实际接收脉冲

- 精确测量上升沿时间差收到距离、接收脉冲失真等影响。

<http://www.f4news.com/2019/12/11/d-tof-lidar-model/>

三维数据的获取方式及原理

➤ 飞行时间TOF法:

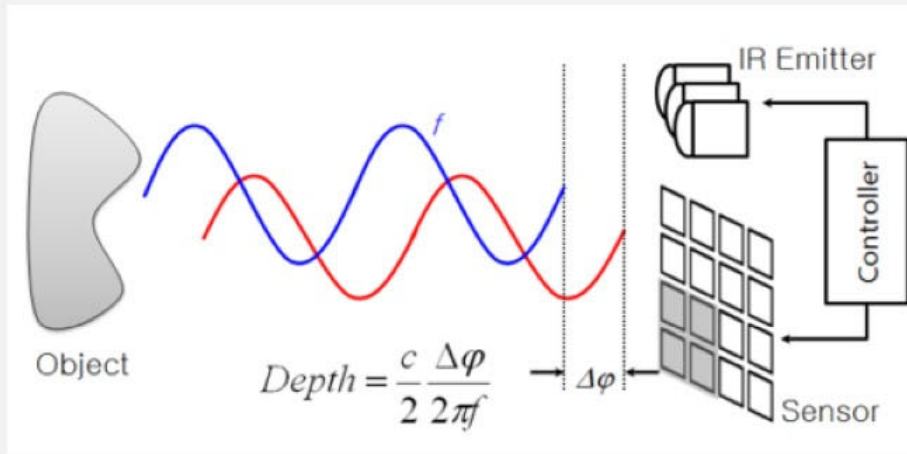


- 用积分比值反映延迟信息: $r = \frac{q_2}{q_1 + q_2}$
 $r = 0$ 对应 $q_2 = 0$, 接收发送信号重合, 没有延迟
 $r = 1$ 对应 $q_1 = 0$, 接收发送信号延迟 T
- 延迟量计算: $\Delta t = T \frac{q_2}{q_1 + q_2}$
- 光飞行距离: $cT \frac{q_2}{q_1 + q_2}$, 考虑光的来回, 物体到相机距离是: $d = \frac{cT}{2} \cdot \frac{q_2}{q_1 + q_2}$
- 为提高精度, 需要多个脉冲重复测量, 将结果平均

$$t_D = 2 \cdot \frac{D}{c} = 2 \cdot \frac{2.5 \text{ m}}{300\,000\,000 \frac{\text{m}}{\text{s}}} = 0.000\,000\,016\,66 \text{ s} = 16.66 \text{ ns}$$

三维数据的获取方式及原理

➤ 飞行时间TOF法:



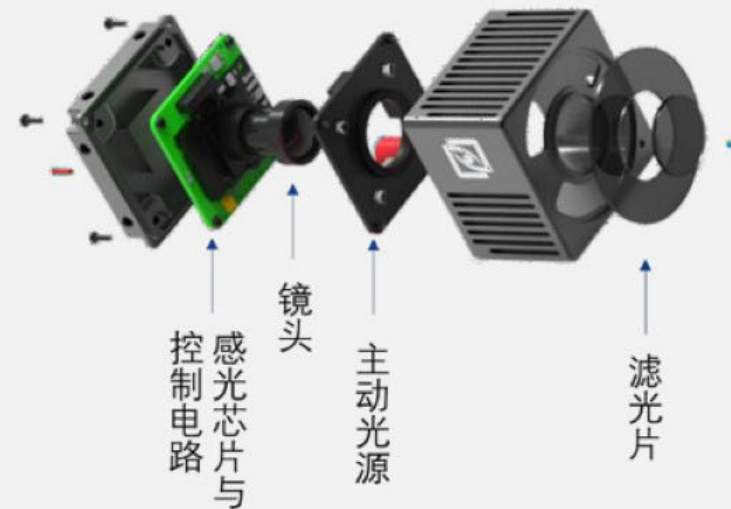
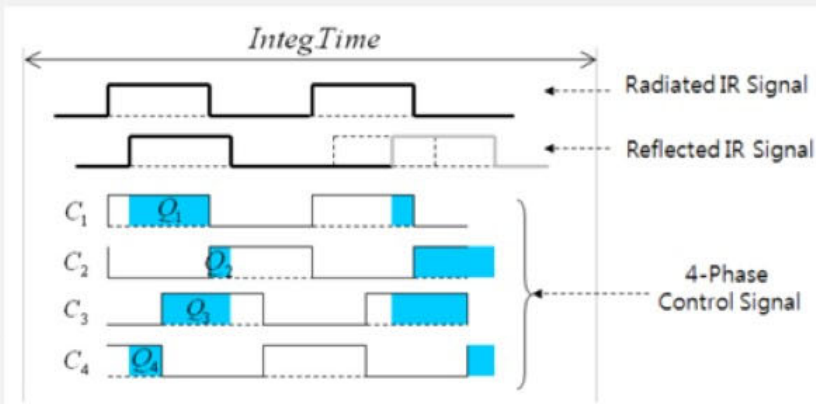
- 接收信号: $r(t) = 1 + a \cos(\omega t - \phi)$

飞行时间带来的相移

- 相移转成时间延迟: $\Delta t = T \frac{\phi}{2\pi}$ (其中T是调制信号周期)
- 光飞行距离: $c\Delta t = cT \frac{\phi}{2\pi} = \frac{c}{F} \frac{\phi}{2\pi}$
- 考虑光的来回, 物体到相机距离是: $d = \frac{c\phi}{4\pi F}$

三维数据的获取方式及原理

➤ 飞行时间TOF法:



• 如何检测相位?

$$\phi = \arctan\left(\frac{Q_3 - Q_4}{Q_1 - Q_2}\right)$$

$$d = \frac{c\phi}{4\pi F}$$

$$A = \frac{\sqrt{(Q_1 - Q_2)^2 + (Q_3 - Q_4)^2}}{2}$$

- 相位测量方法: 通过积分器得到接收信号 $r(t)$ 和若干特定参考信号 $g_\theta(t) = \cos(\omega t + \theta)$ 的相关值
- 相关计算公式: $s_\theta = \lim_{t \rightarrow \infty} \frac{1}{t} \int_{-\frac{t}{2}}^{\frac{t}{2}} r(t) g_\theta(t) dt = \frac{a}{2} \cos(\phi + \theta)$
- 设置参考信号 $g_\theta(t)$ 和发射信号相差 θ 分别为: $0, 0.5\pi, \pi, 1.5\pi$, 送入相关电路分别得到:
 $s_0 = \frac{a}{2} \cos(\phi)$ 、 $s_{0.5\pi} = -\frac{a}{2} \sin(\phi)$ 、 $s_\pi = -\frac{a}{2} \cos(\phi)$ 、 $s_{1.5\pi} = \frac{a}{2} \sin(\phi)$
- 上面对相关值分析表明
相位差: $\phi = \text{atan}\left(\frac{s_{1.5\pi} - s_{0.5\pi}}{s_0 - s_\pi}\right)$ 以及 接收信号强度: $a = \frac{\sqrt{(s_{1.5\pi} - s_{0.5\pi})^2 + (s_0 - s_\pi)^2}}{2}$

<https://hal.inria.fr/hal-00725654/document>

<https://www.mouser.cn/applications/time-of-flight-robotics/>

三维数据的获取方式及原理

➤ RGBD融合:

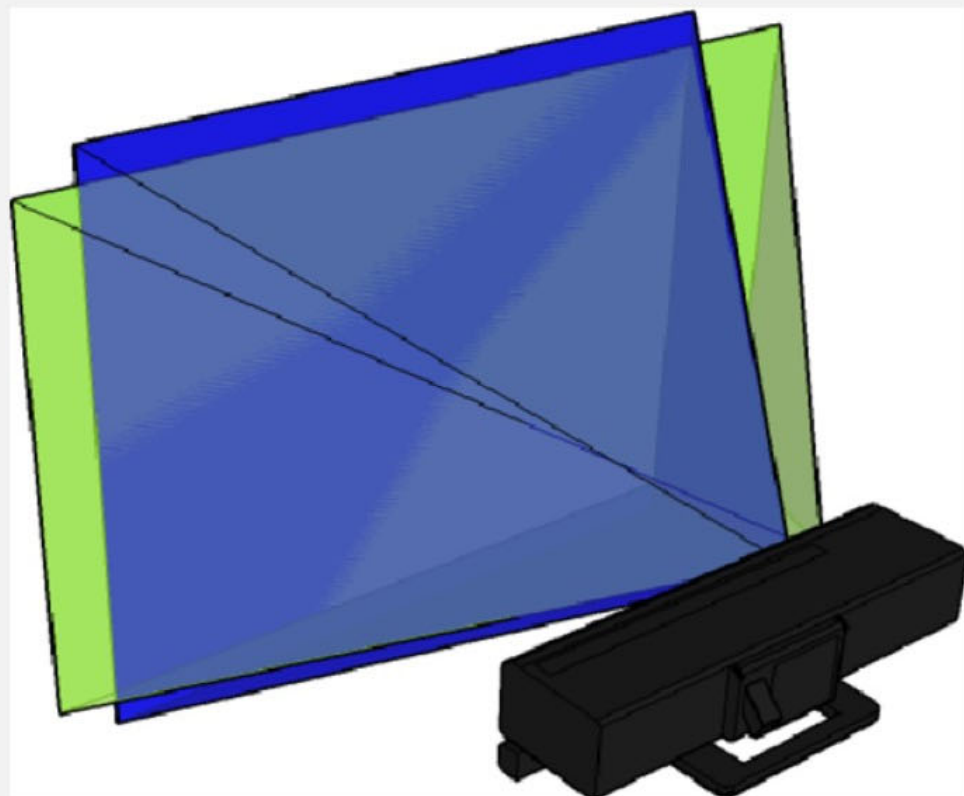
RGB相机中的像素位置
(和TOF相机坐标系
下的点云对应)

TOF相机坐标系
下的点云坐标

$$Z \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix} \begin{bmatrix} t_x \\ t_y \\ t_z \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

RGB相机的
内参

TOF相机坐标系转到
RGB相机坐标系





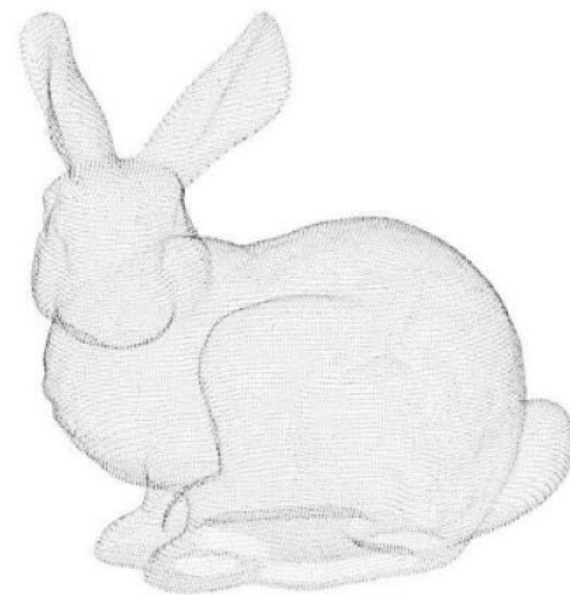
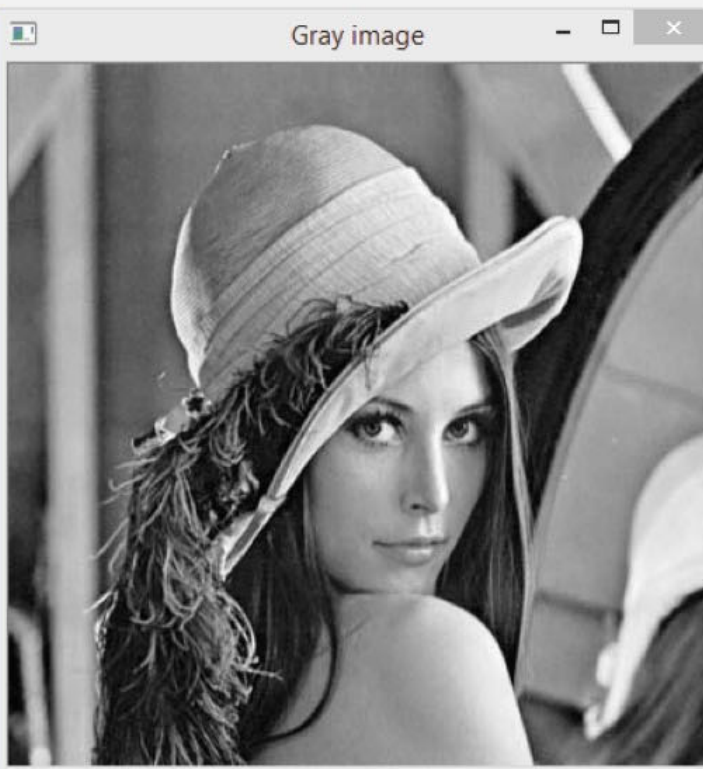
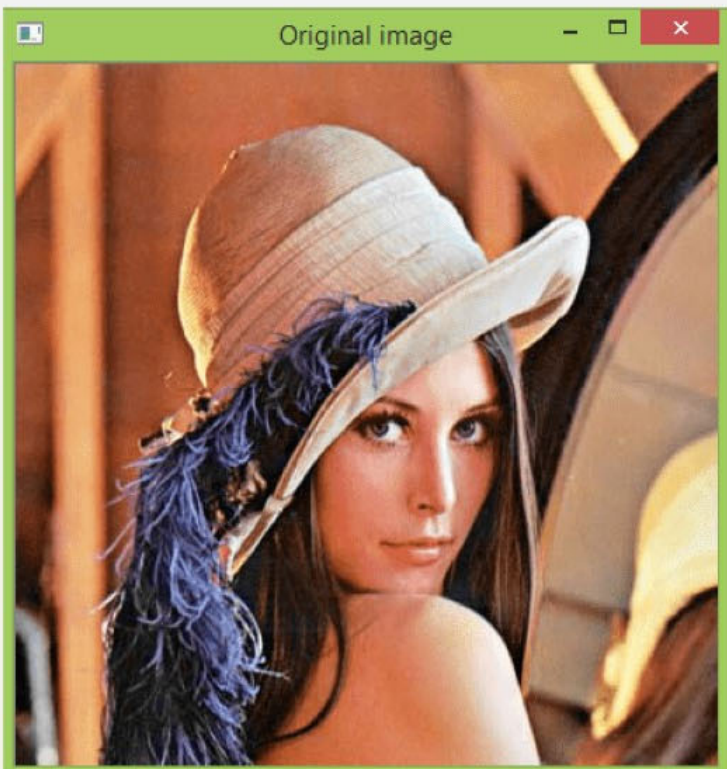
内容概要

三维数据的获取方式及原理

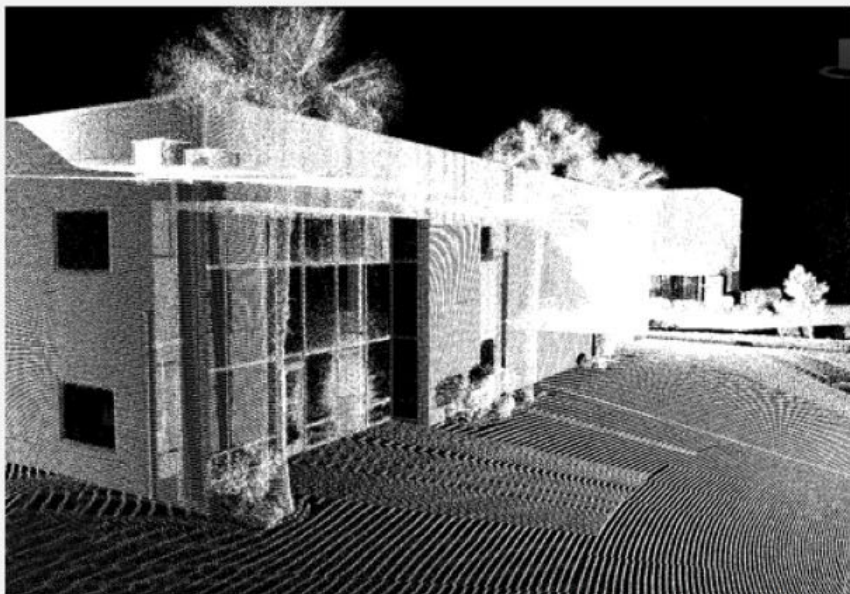
三维信息的表征形式

点云的基本特征和描述

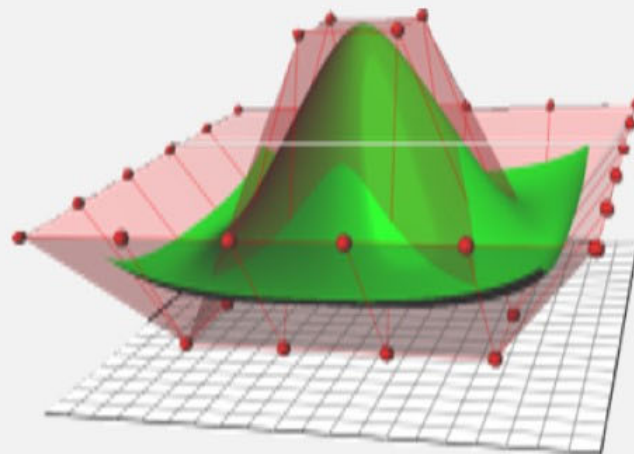
三维信息的表征形式



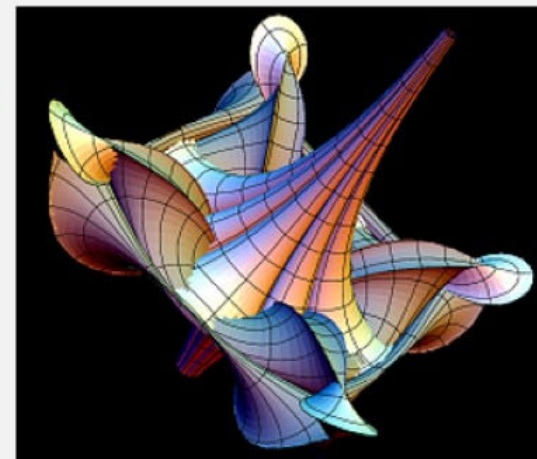
三维信息的表征形式



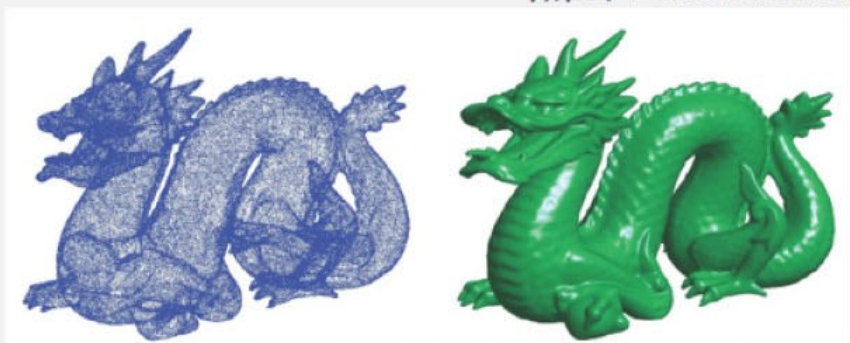
点云 PointCloud



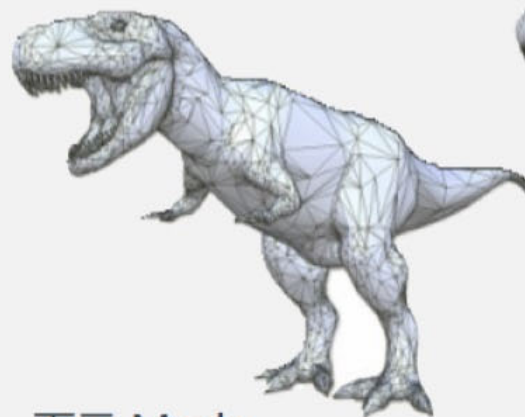
参数化曲面 Parametric surfaces



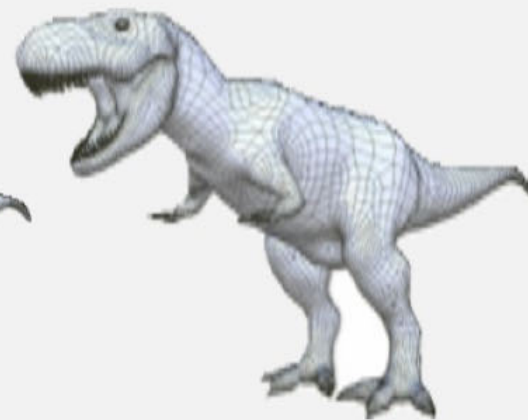
隐式曲面 Implicit surface



细分曲面 Subdivision surfaces



面元 Mesh



三维信息的表征形式

➤ 三维表征:

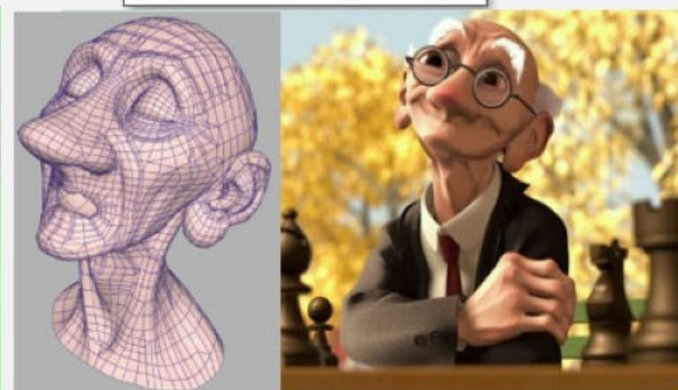
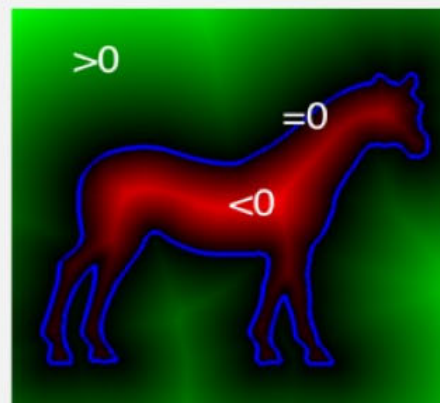
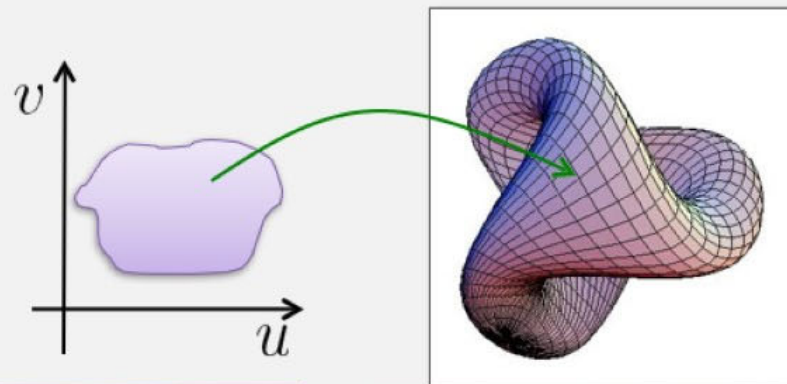
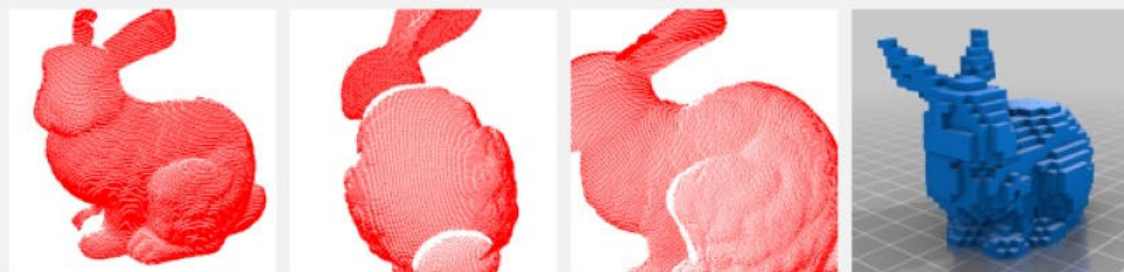
点云

参数化曲线曲面

隐式曲线曲面

多边形面元

体素

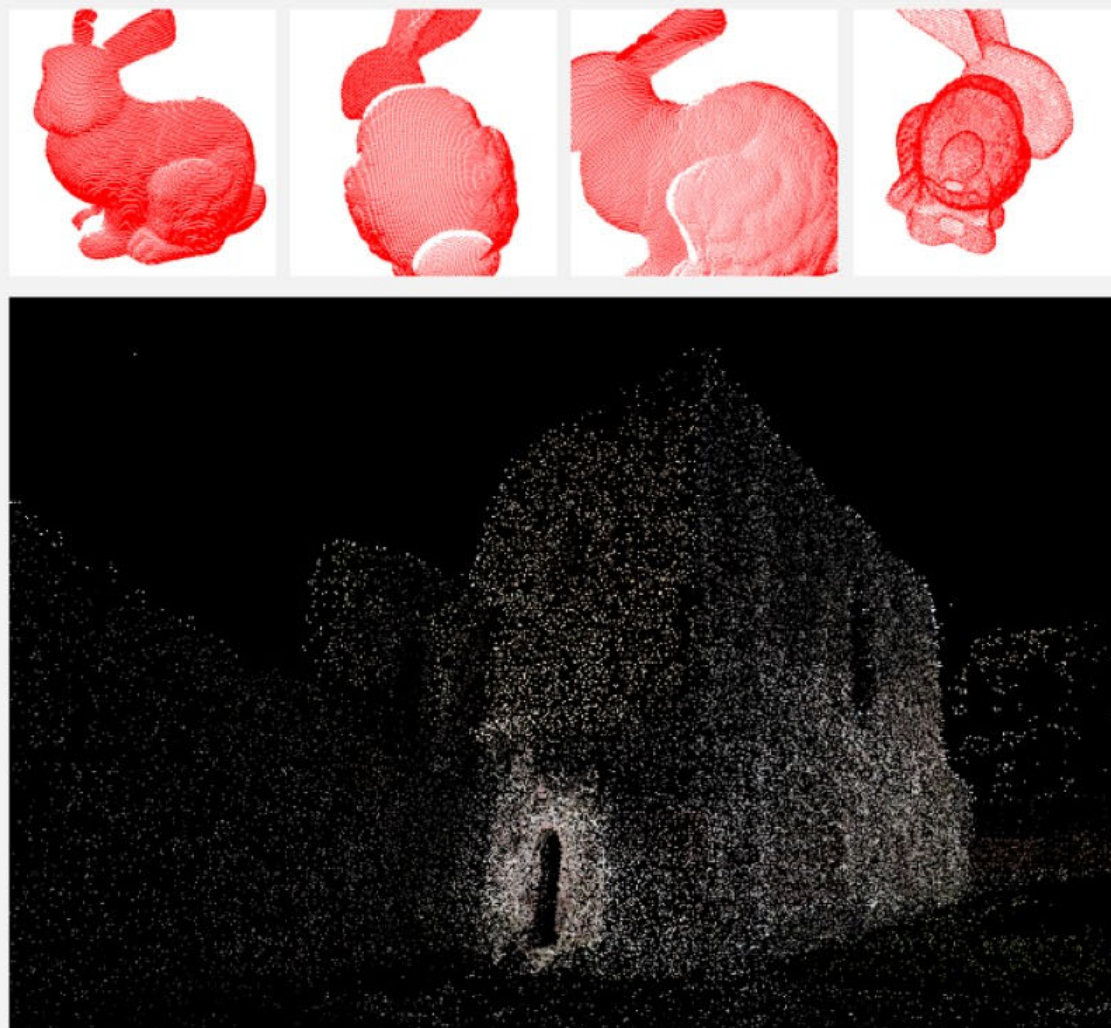


三维信息的表征形式

➤ 点云:

$$P = \{p_0, \dots, p_N \mid p_i \in \mathbb{R}^3\}$$

- 点云是点组成的集合，基本组成是 (x,y,z)
- 容易处理、编辑
- 可以转化到多种表征：面元、隐式表面、参数化表面
- 高效的点云处理和建模需要空间分区数据结构，例如点云的邻域。
- 点云的不规则、稀疏性、局部性



三维信息的表征形式

➤ 参数化曲线曲面:

$$f: X \rightarrow Y, X \subseteq \mathbb{R}^m, Y \subseteq \mathbb{R}^n$$

平面曲线: $m = 1, n = 2$

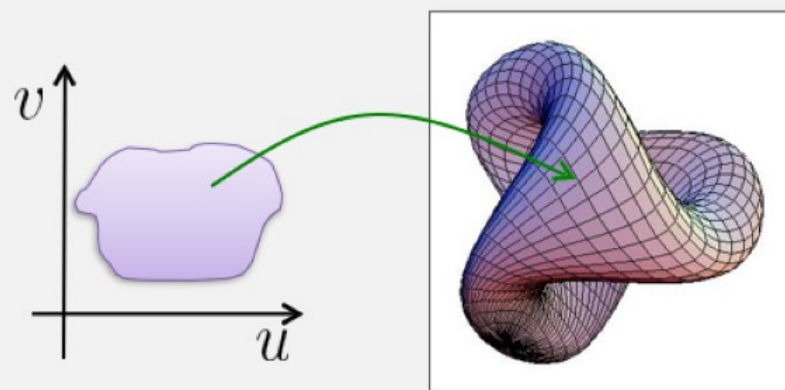
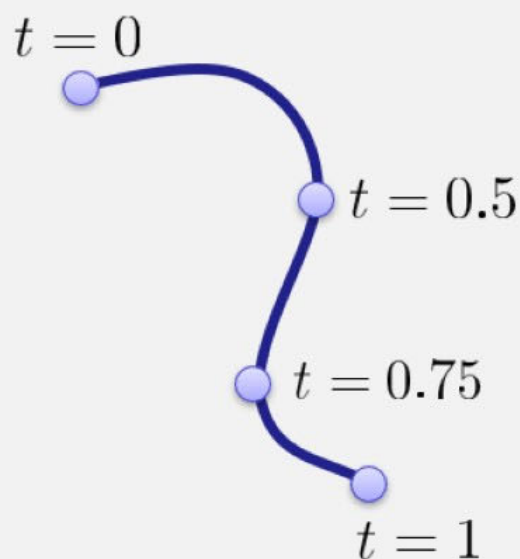
$$s(t) = (x(t), y(t))$$

空间曲线: $m = 1, n = 3$

$$s(t) = (x(t), y(t), z(t))$$

空间曲面: $m = 2, n = 3$

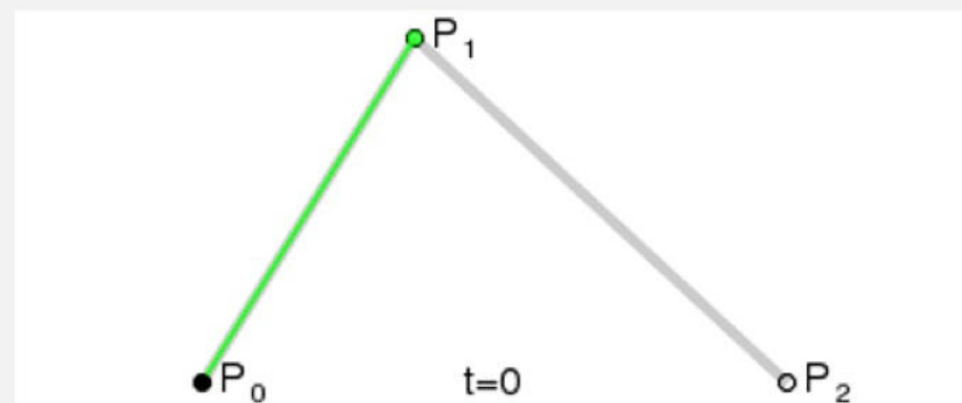
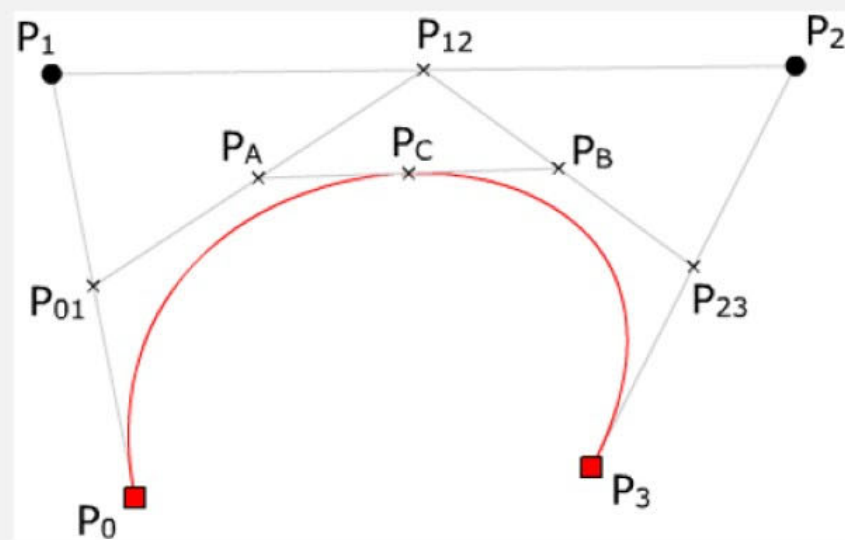
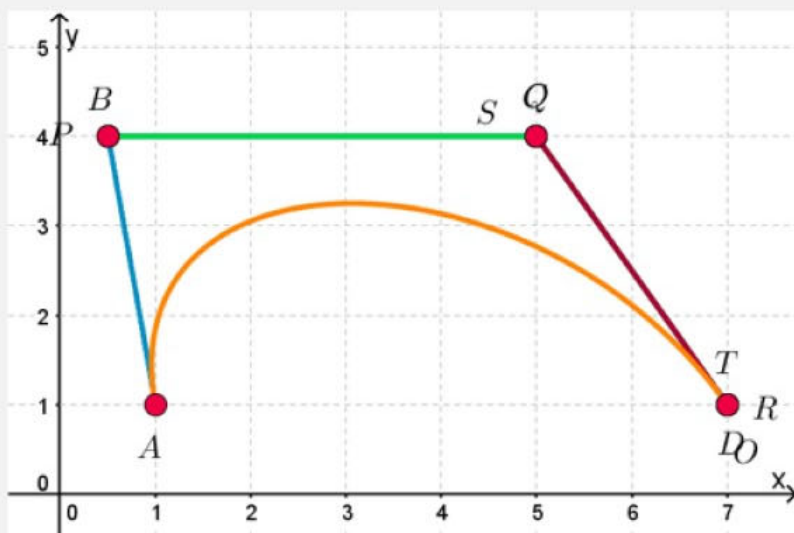
$$s(u, v) = (x(u, v), y(u, v), z(u, v))$$



三维信息的表征形式

- 参数化曲线:
贝塞尔曲线, 样条线:

$$s(t) = \sum_{i=0}^n \mathbf{p}_i B_i^n(t) \quad B_i^n(t) = \binom{n}{i} t^i (1-t)^{n-i}$$



三维信息的表征形式

➤ 参数化曲面:

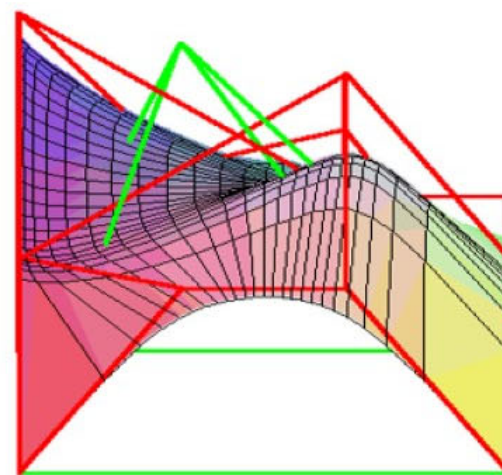
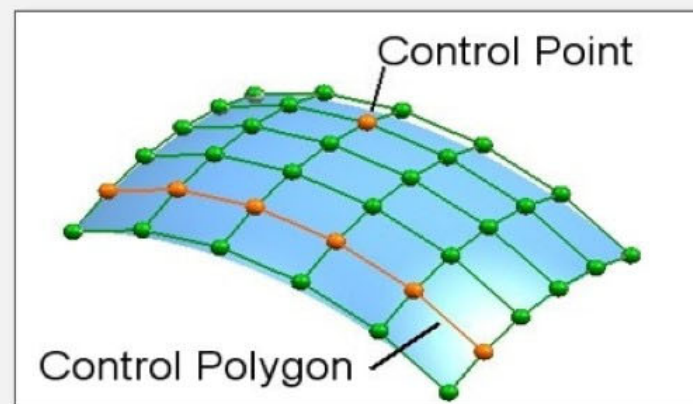
曲线扫描:

$$s(u, v) = \sum_{i,j} \mathbf{p}_{i,j} B_i(u) B_j(v)$$

贝塞尔曲面:

$$s(u, v) = \sum_{i=0}^m \sum_{j=0}^n \mathbf{p}_{i,j} B_i^m(u) B_j^n(v)$$

- **优点:**
- 容易生成曲线或者曲面上的点
- 参数分离, 容易分析
- **缺点:**
- 难以确定内部外部空间
- 难以判断点是否在曲面曲线上
- 难以生成非常复杂的曲线曲面 -> 局部表面 (多边形面元)



三维信息的表征形式

➤ 隐式曲线曲面:

显式曲面: 形如 $z=f(x, y)$

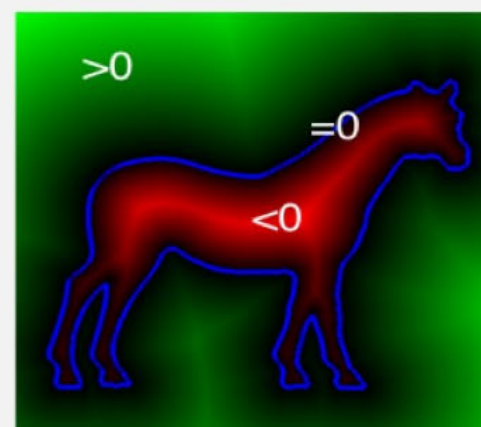
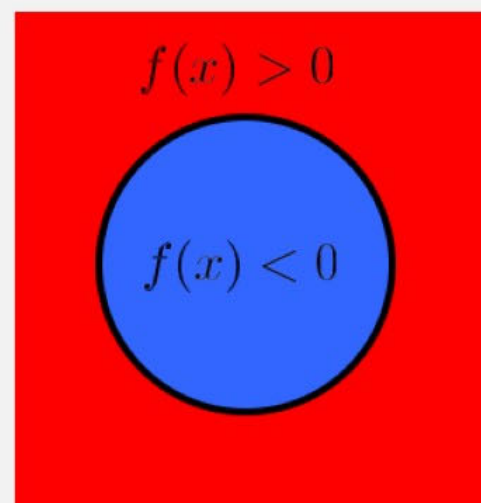
隐式曲面: 形如 $F(x, y, z)=0$

水平集 (Level-set) 函数: $f: \mathbb{R}^m \rightarrow \mathbb{R}$

平面曲线: $S = \{x \in \mathbb{R}^2 | f(x) = 0\}$

空间曲面: $S = \{x \in \mathbb{R}^3 | f(x) = 0\}$

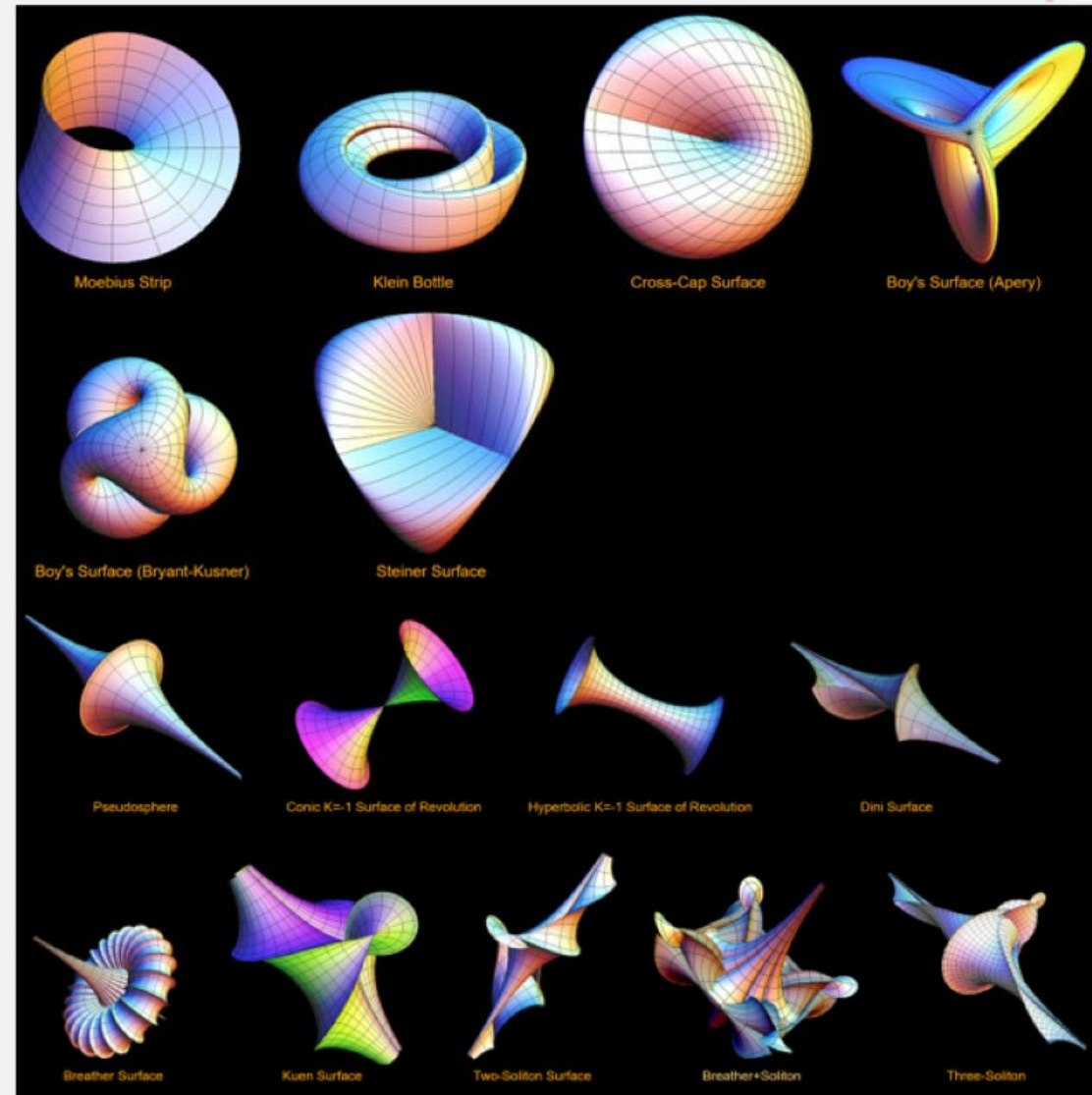
空间分区: $\{x \in \mathbb{R}^m | f(x) > 0\}$ 外部
 $\{x \in \mathbb{R}^m | f(x) = 0\}$ 曲面/曲线
 $\{x \in \mathbb{R}^m | f(x) < 0\}$ 内部



三维信息的表征形式

➤ 隐式曲线曲面:

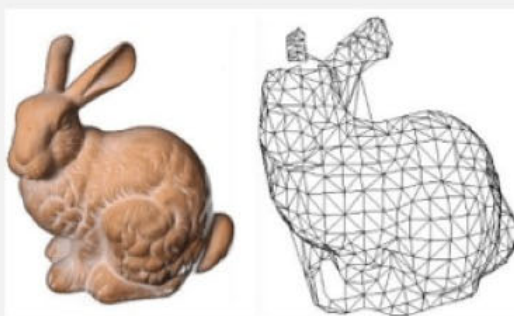
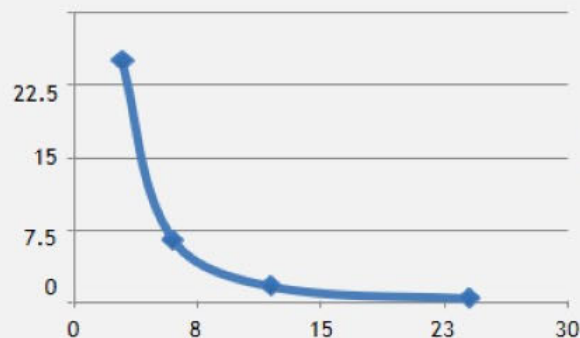
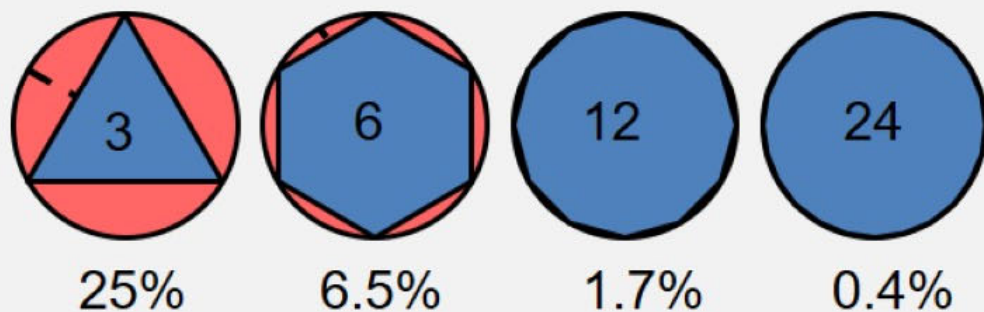
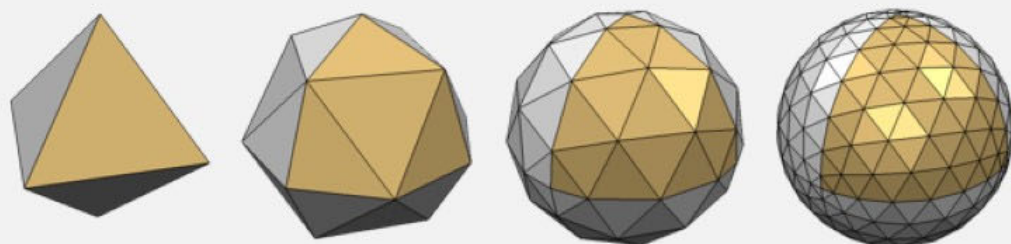
- **优点:**
- 容易确定内部外部空间
- 容易确定点是否在曲线曲面上
- **缺点:**
- 难以生成曲线曲面上的点
- 不适合实时渲染



http://virtualmathmuseum.org/Surface/gallery_o.html

三维信息的表征形式

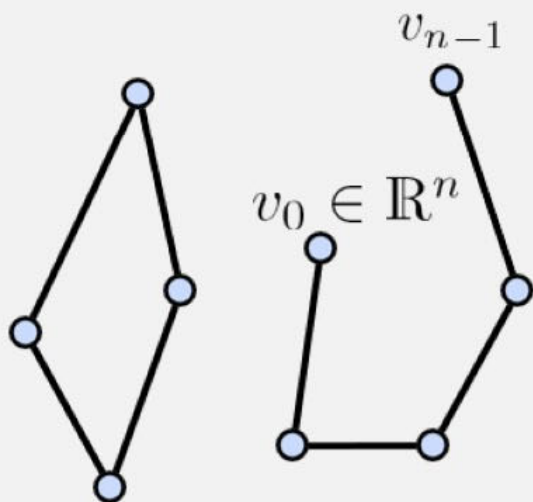
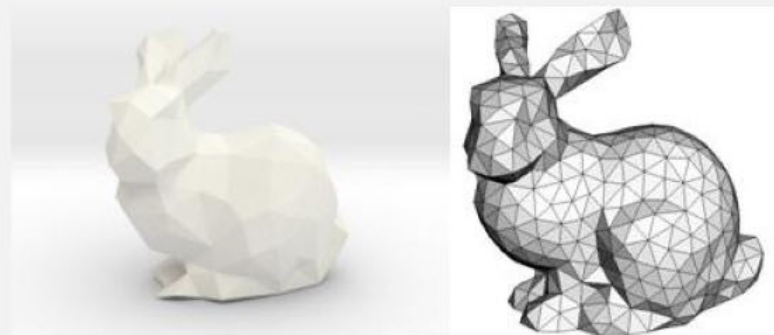
➤ 多边形面元:



- 量化误差可控
- 应用于任意拓扑结构
- 支持动态调整
- 可以利用GPU高效运算
- 多边形面元vs三角形面元

三维信息的表征形式

➤ 多边形:



顶点: $v_0, v_1, \dots, v_{n-1}$

边: $\{(v_0, v_1), \dots, (v_{n-2}, v_{n-1})\}$

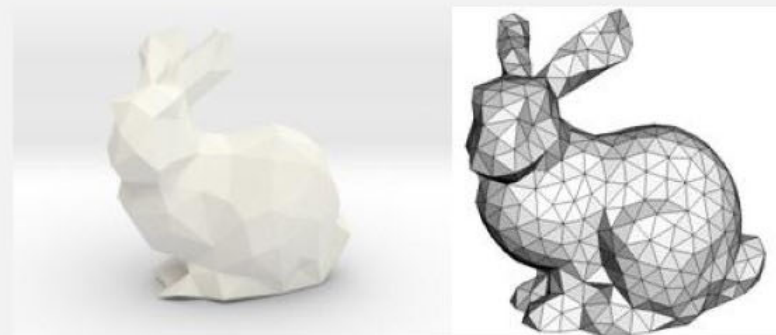
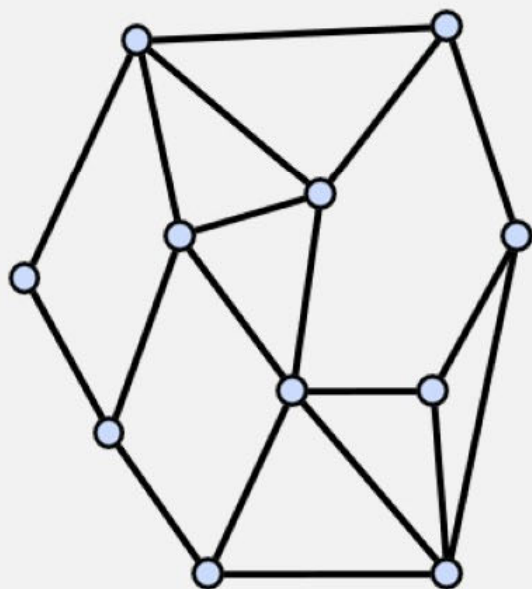
闭合性: $v_0 = v_{n-1}$

共面性: 所有的顶点在一个平面

单一性: 不存在交叉

三维信息的表征形式

➤ 多边形面元:



- 多边形面元: 有闭合的、单一的多边形 Q_i 组成的有限集 M
- 在 M 中, 任意两个多边形的交点要么为空, 要么是一个顶点或者一条边

$$M = \langle V, E, F \rangle$$

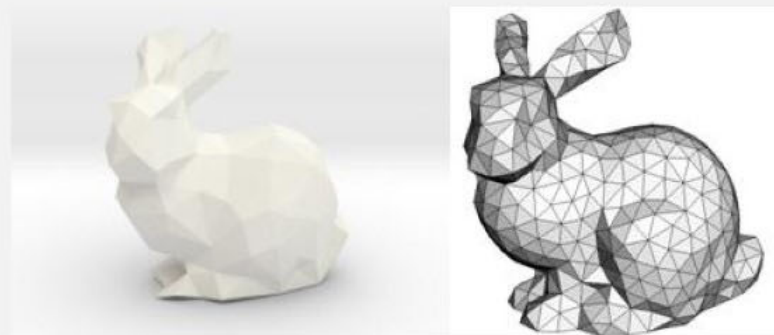
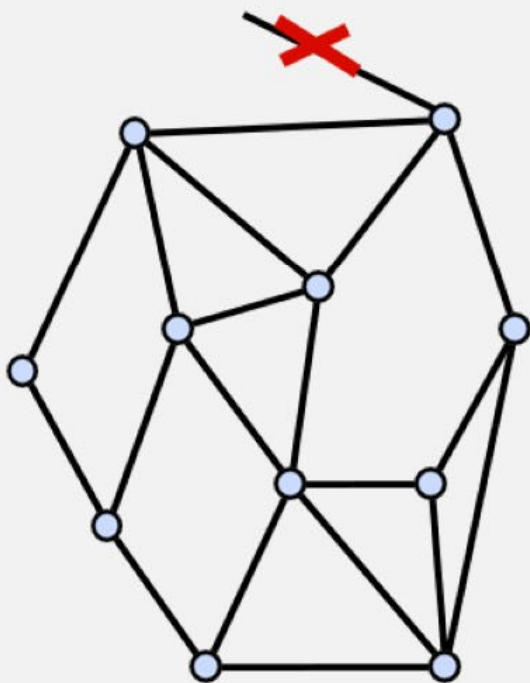
顶点
vertices

边
edges

面
faces

三维信息的表征形式

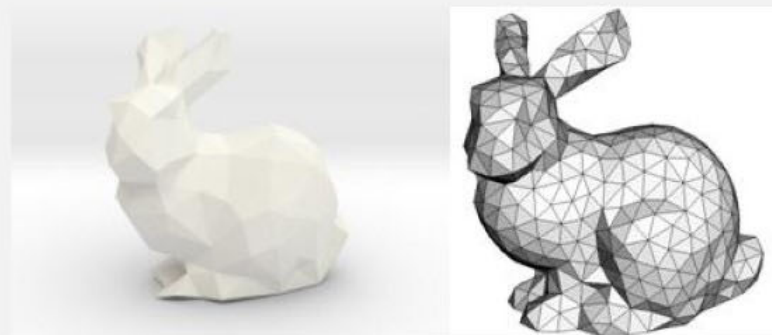
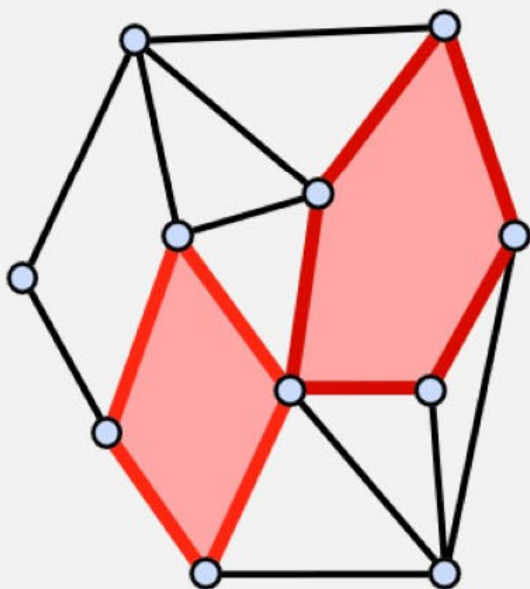
➤ 多边形面元:



- 多边形面元: 有闭合的、单一的多边形 Q_i 组成的有限集 M
- 在 M 中, 任意两个多边形的交点要么为空, 要么是一个顶点或者一条边
- 每条边至少属于一个多边形

三维信息的表征形式

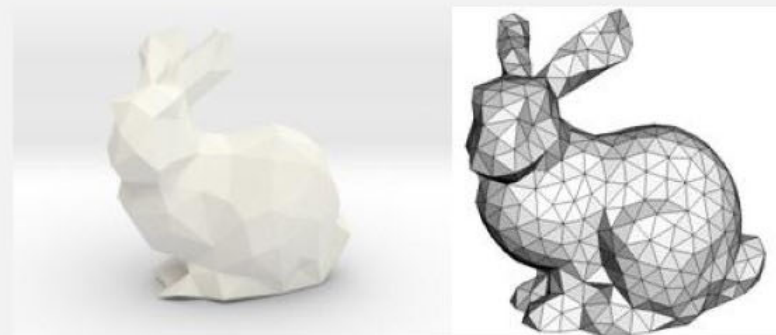
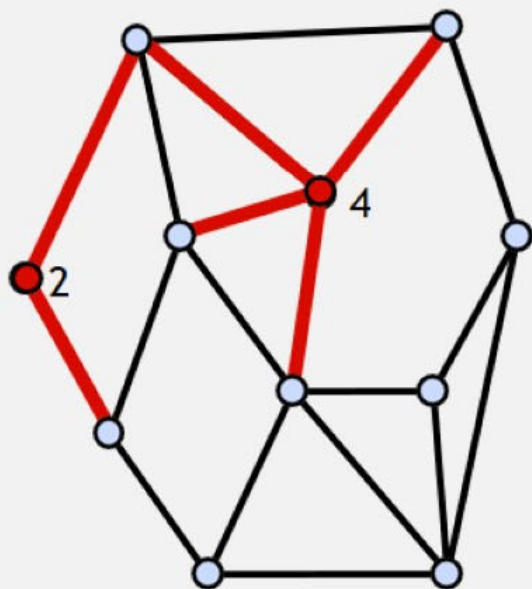
➤ 多边形面元:



- 多边形面元: 有闭合的、单一的多边形 Q_i 组成的有限集 M
- 在 M 中, 任意两个多边形的交点要么为空, 要么是一个顶点或者一条边
- 每条边至少属于一个多边形
- 每个 Q_i 定义了多边形网格的一个面

三维信息的表征形式

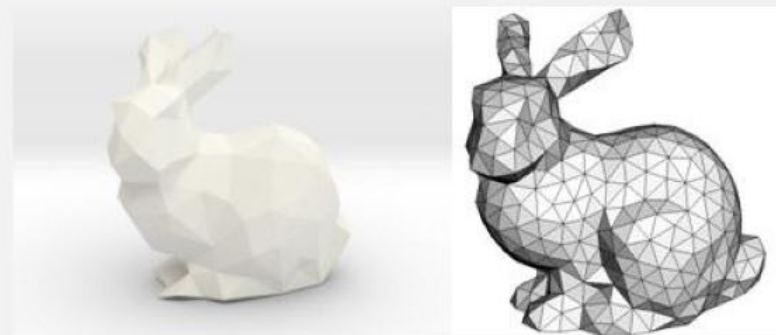
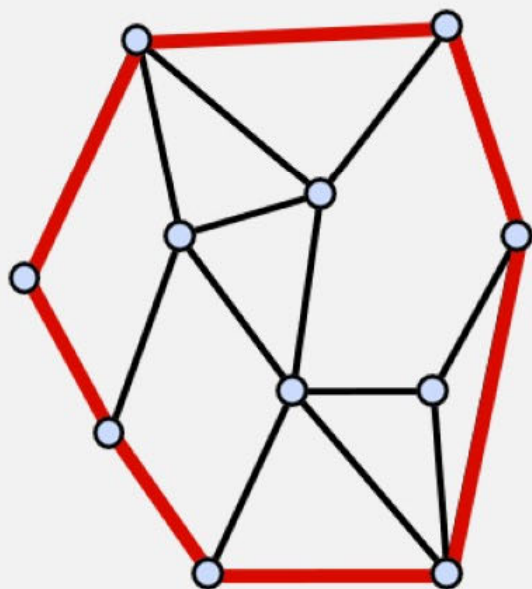
➤ 多边形面元:



- 多边形面元: 有闭合的、单一的多边形 Q_i 组成的有限集 M
- 在 M 中, 任意两个多边形的交点要么为空, 要么是一个顶点或者一条边
- 每条边至少属于一个多边形
- 每个 Q_i 定义了多边形网格的一个面
- 顶点的度=入射边的数目

三维信息的表征形式

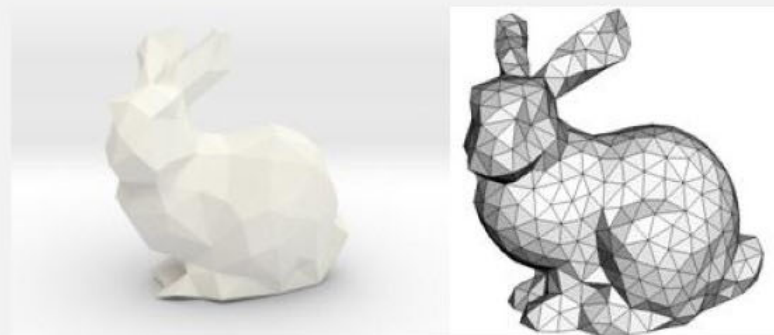
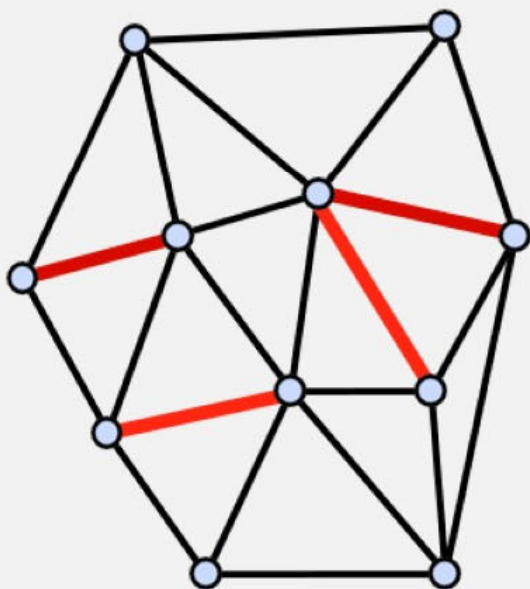
➤ 多边形面元:



- 多边形面元: 有闭合的、单一的多边形 Q_i 组成的有限集 M
- 在 M 中, 任意两个多边形的交点要么为空, 要么是一个顶点或者一条边
- 每条边至少属于一个多边形
- 每个 Q_i 定义了多边形网格的一个面
- 顶点的度=入射边的数目
- 多边形面元的边界

三维信息的表征形式

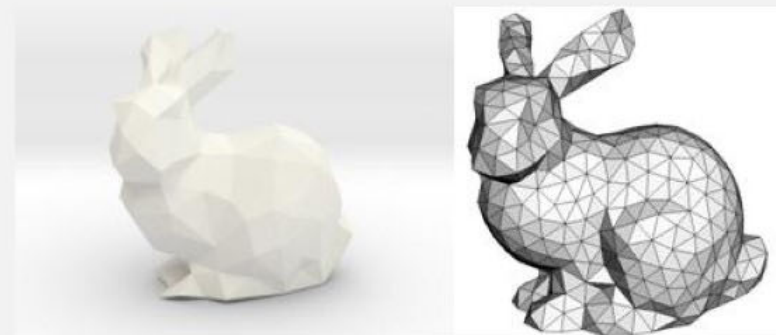
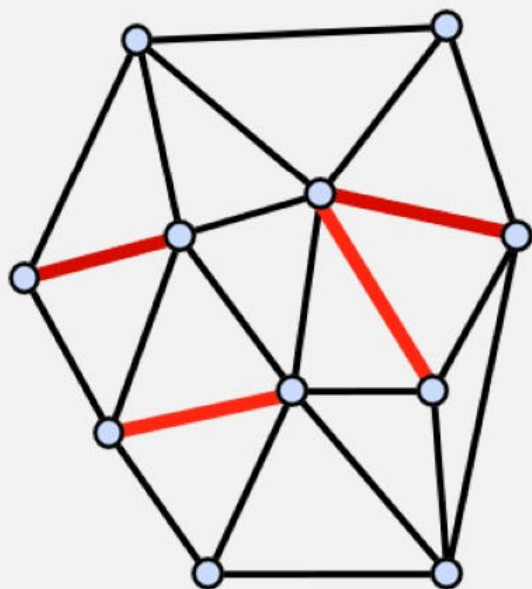
➤ 三角形面元:



- 多边形面元，其中每个面是三角形
- 简化数据结构
- 简化渲染
- 简化算法
- 每个面都是平面和凸的
- 任何多边形都可以三角化

三维信息的表征形式

➤ 三角形面元:



$$V = \{v_1, \dots, v_n\}$$

$$E = \{e_1, \dots, e_k\}, \quad e_i \in V \times V$$

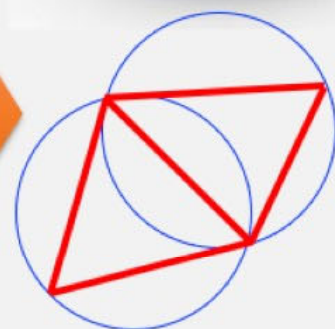
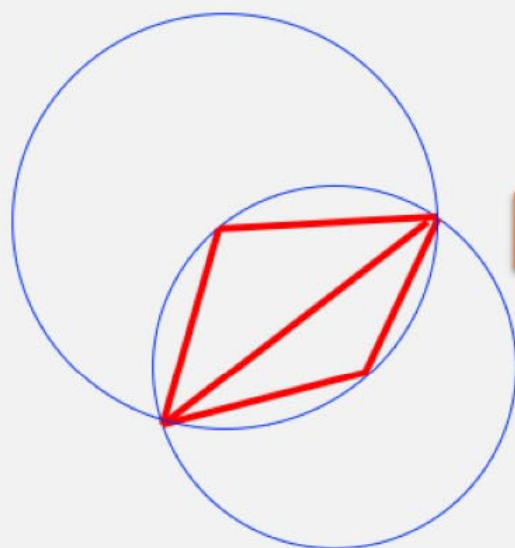
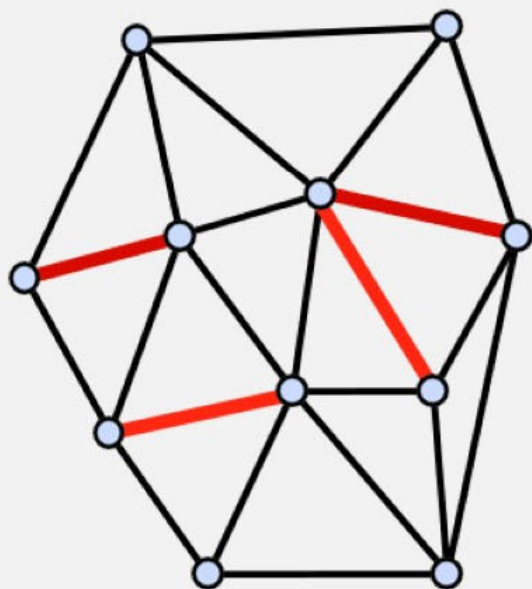
$$F = \{f_1, \dots, f_m\}, \quad f_i \in V \times V \times V$$

$$P = \{\mathbf{p}_1, \dots, \mathbf{p}_n\}, \quad \mathbf{p}_i \in \mathbb{R}^3$$

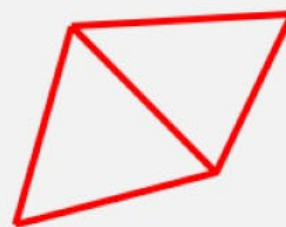
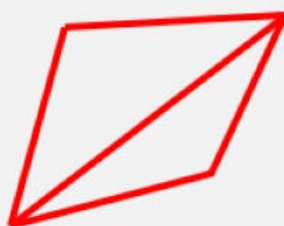
- Delaunay三角剖分
- Bowyer-Watson算法

三维信息的表征形式

➤ 三角形面元:

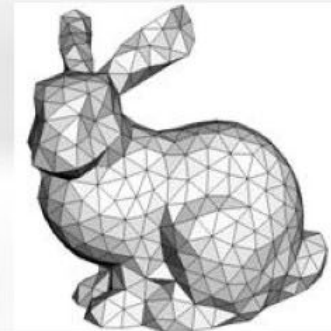
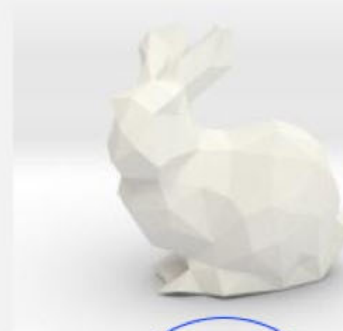


1. 空圆



2. 最大化最小角

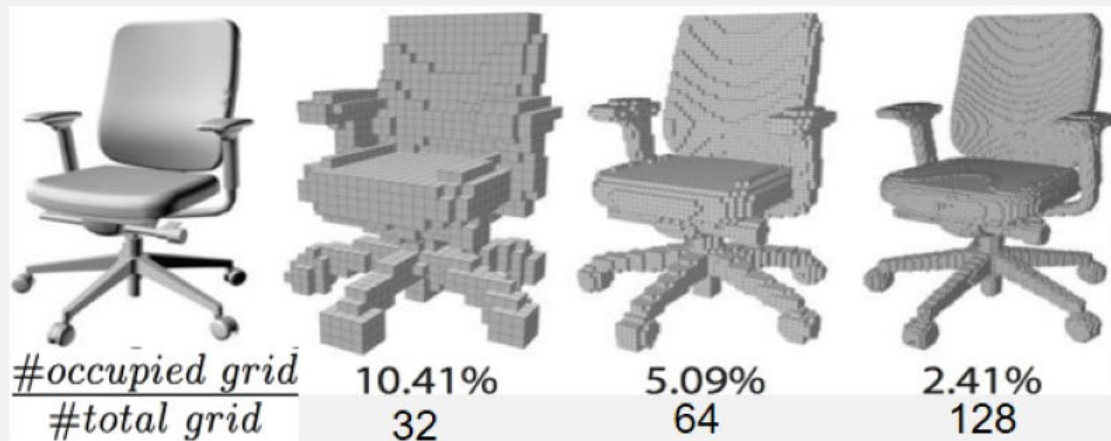
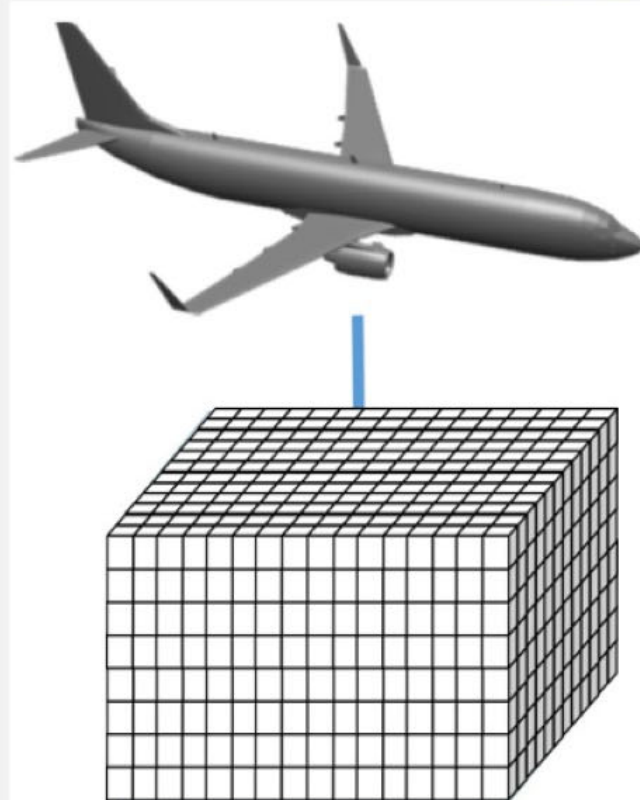
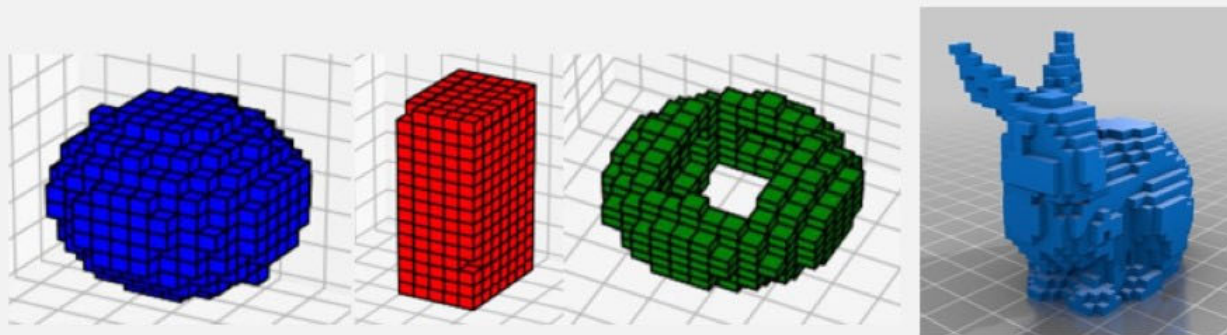
- Delaunay三角剖分准则
- Bowyer-Watson算法



三维信息的表征形式

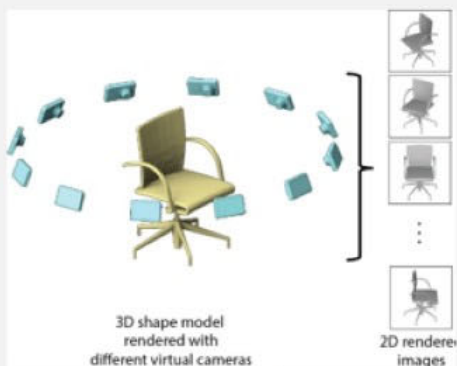
➤ 体素:

- 无序点云有序化
- 空间体素化, 占用格点 0, 1
- 快速检测空间点是否被占据
- 体像素之间的逻辑运算
- 运算速度快但存储效率很低, 特别在深度学习模型中问题突出



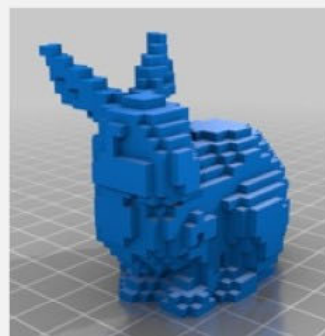
三维信息的表征形式

➤ 深度学习中的三维表征:



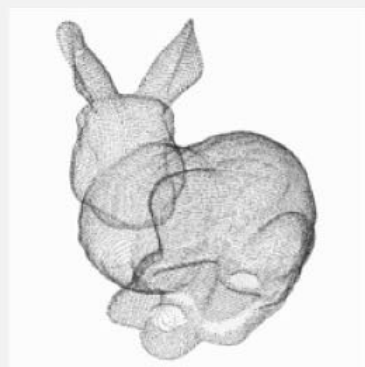
多视图

[Socher et al. 2012]
[Su et al. 2015]
[Wei et al. 2016]
[Kalogerakis et al. 2016]



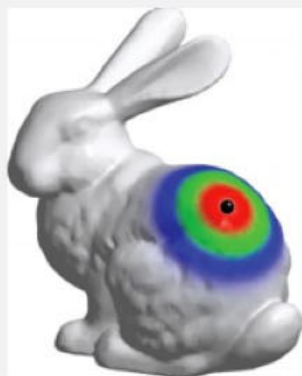
体素化

[Maturana et al. 2015]
[Wu et al. 2015]
[Qi et al. 2016]



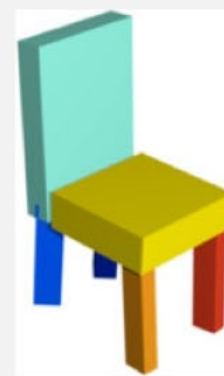
点云

[Qi et al. 2017]
[Fan et al. 2017]



图网络

[Defferrard et al. 2016]
[Henaff et al. 2015]
[Yi et al. 2016]



组合部件

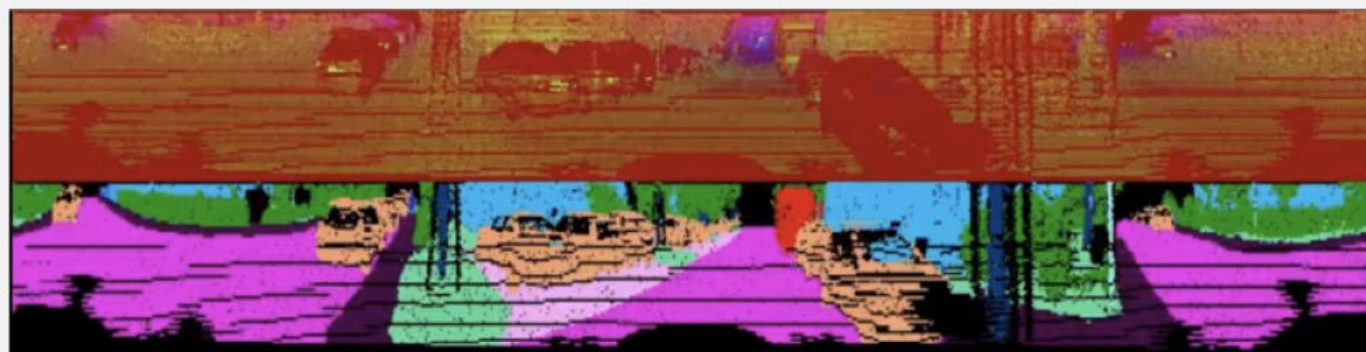
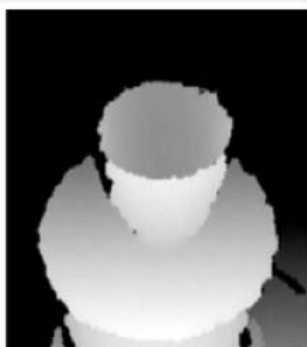
[Tulsiani et al. 2017]

三维信息的表征形式

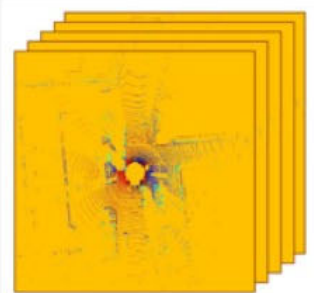
➤ 深度学习中的三维表征:



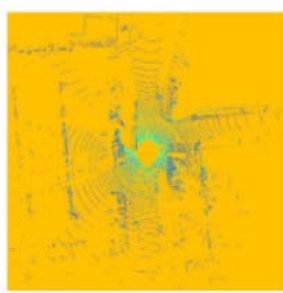
RGBD



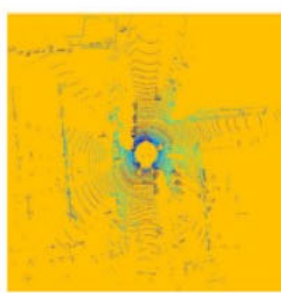
深度图



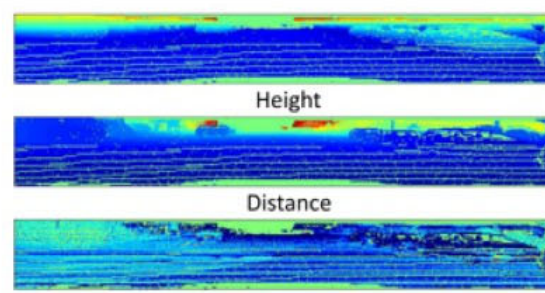
Height Maps



Density



Intensity

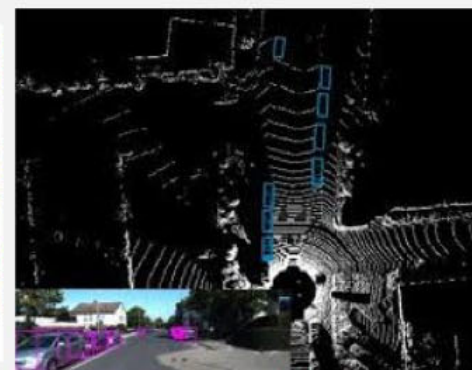


Height

Distance

Intensity

前视图系列



点云鸟瞰图

[Chen et al. 2017]

鸟瞰图系列



内容概要

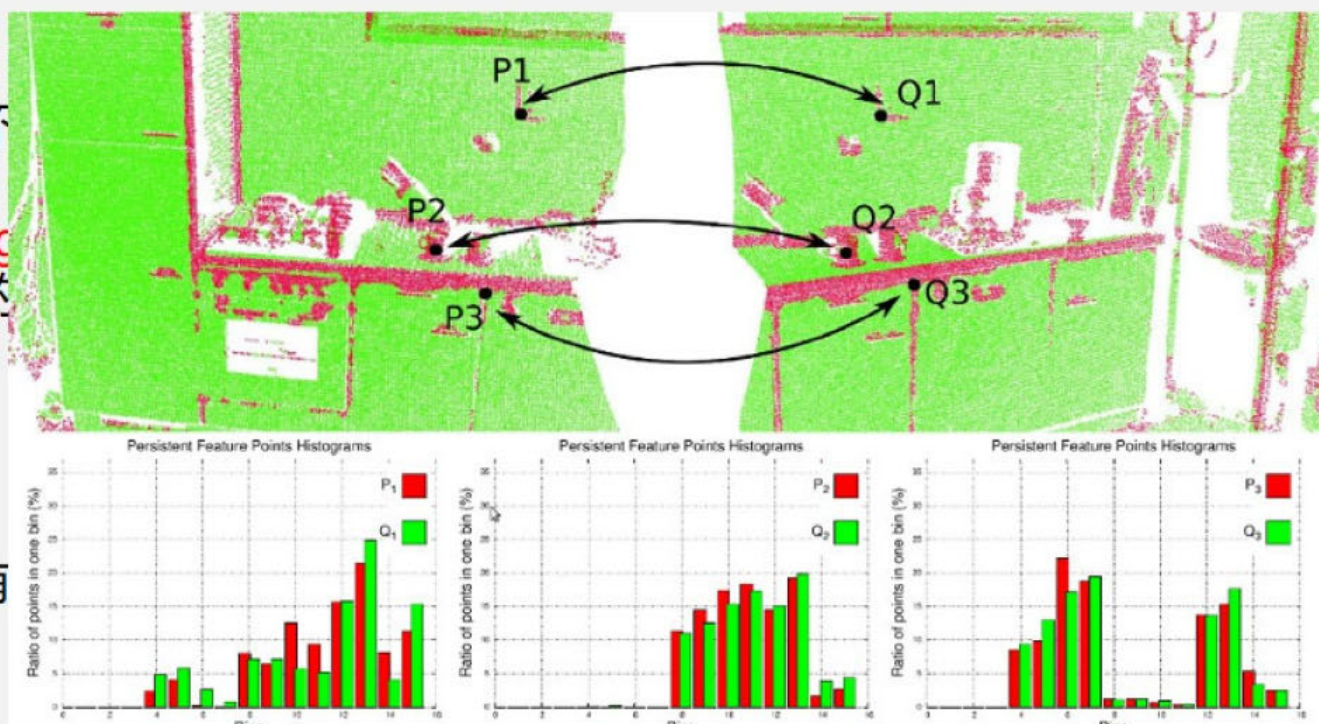
三维数据的获取方式及原理
三维信息的表征形式
点云的基本特征和描述

点云的基本特征和描述

➤ 点云的基本特征:

数据与特征决定了机器学习的上限，而后面的模型/算法/参数只是来逼近这个上限

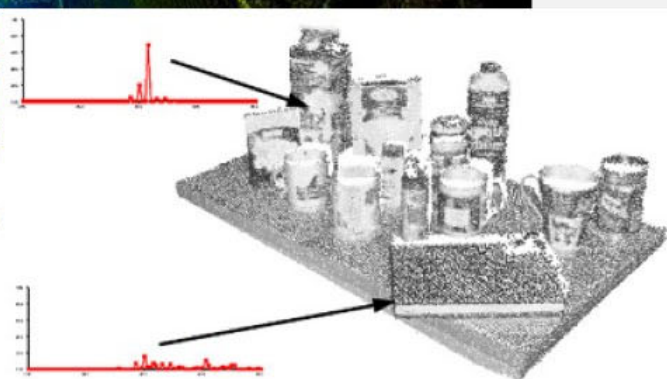
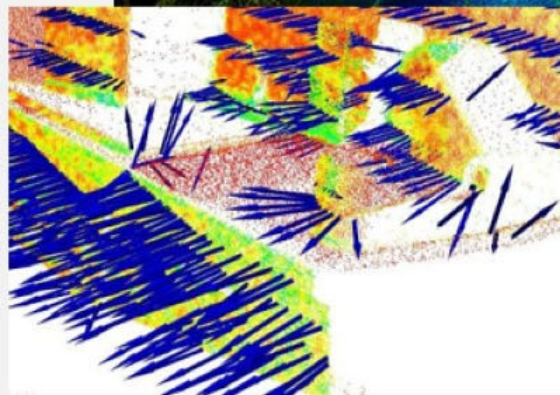
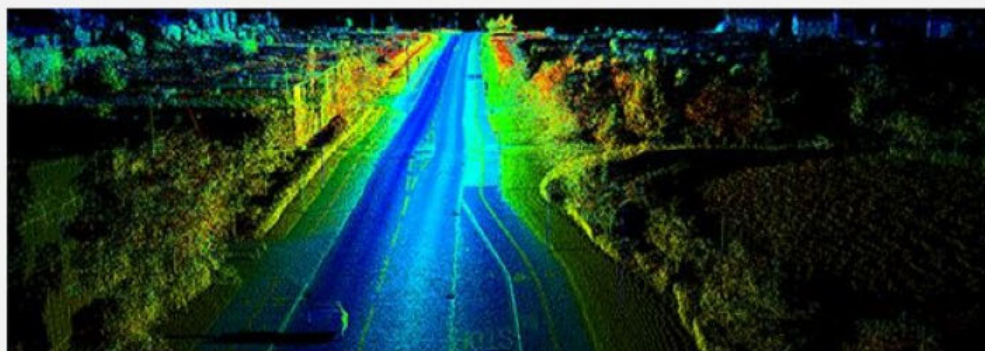
- 刚体变换(rigid transformations)
——即三维旋转和三维平移变化不
- 改变采样密度 (varying sampling)
——原则上，一个局部表面小块的
即特征向量具有抗密度干扰性；
- 噪音 (noise)
——数据中有轻微噪音的情况下，
的值，即特征向量对点云噪声具有



点云的基本特征和描述

➤ 点云的基本特征:

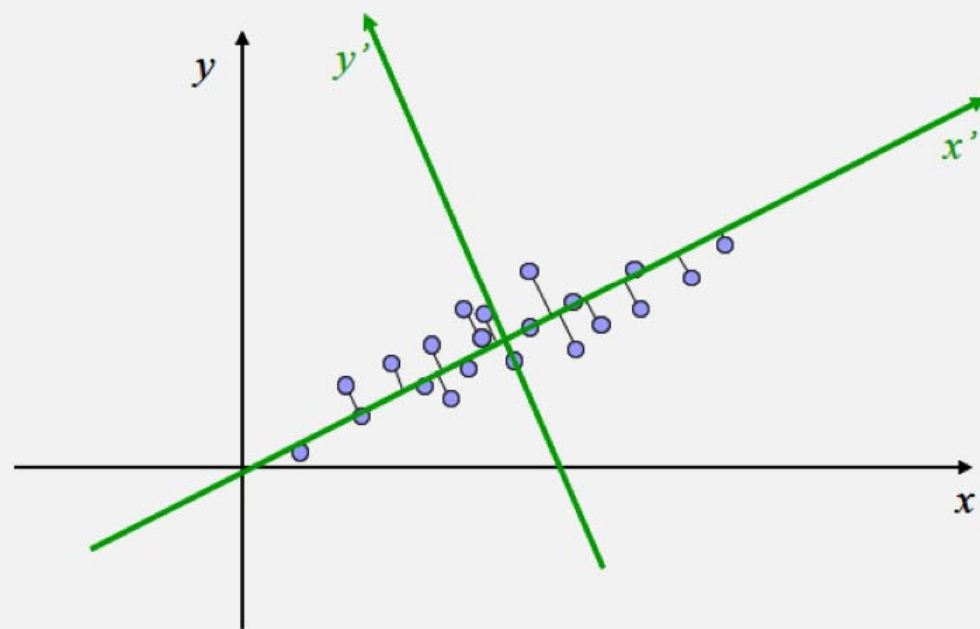
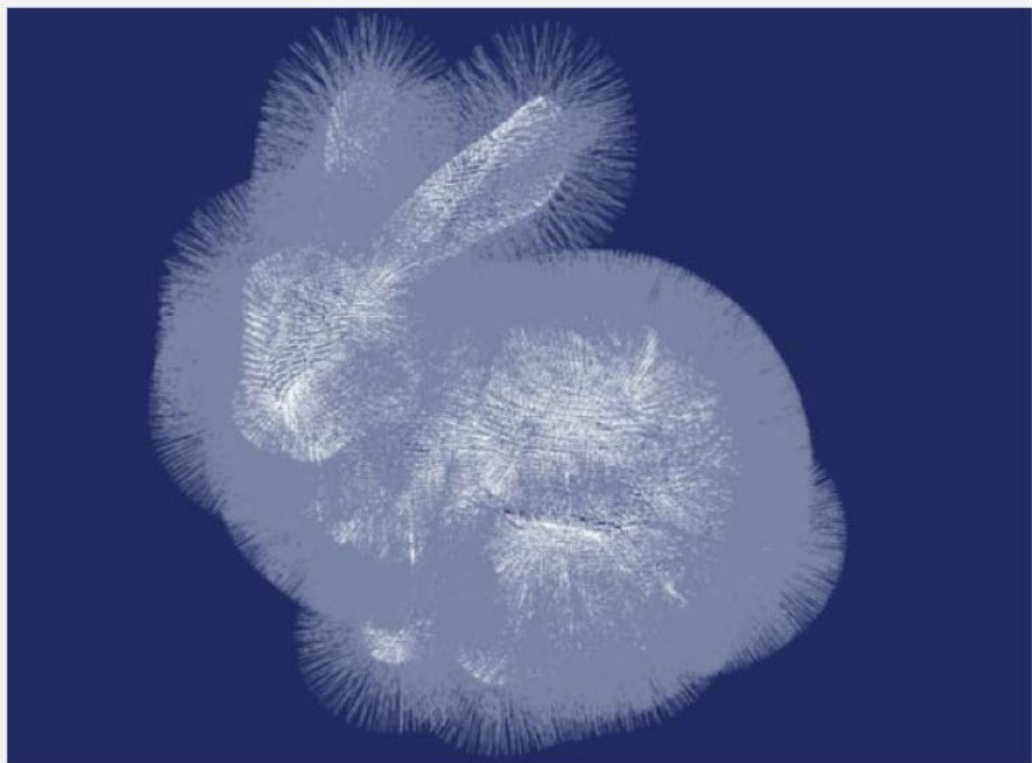
- 按照特征的物理属性, 可以将特征分为: 几何域, 强度域
- 按照特征的空间尺度, 可以分为: 单点特征, 局部特征, 全局特征



<https://blog.csdn.net/shaozhenghan/article/details/81346585>

点云的基本特征和描述

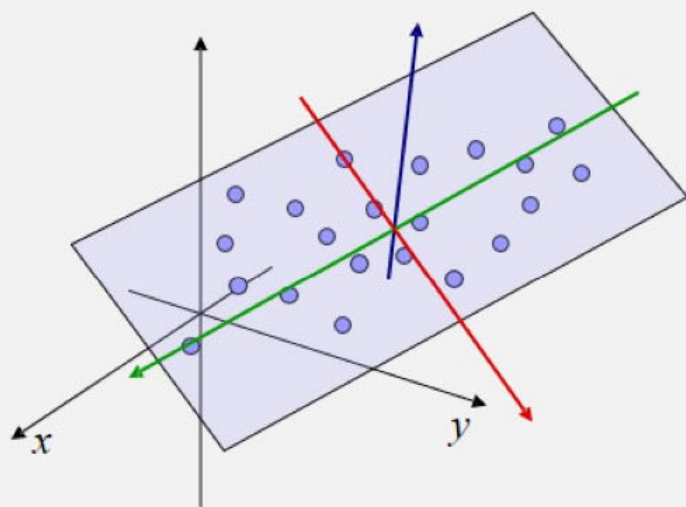
➤ 点云的基本特征描述:



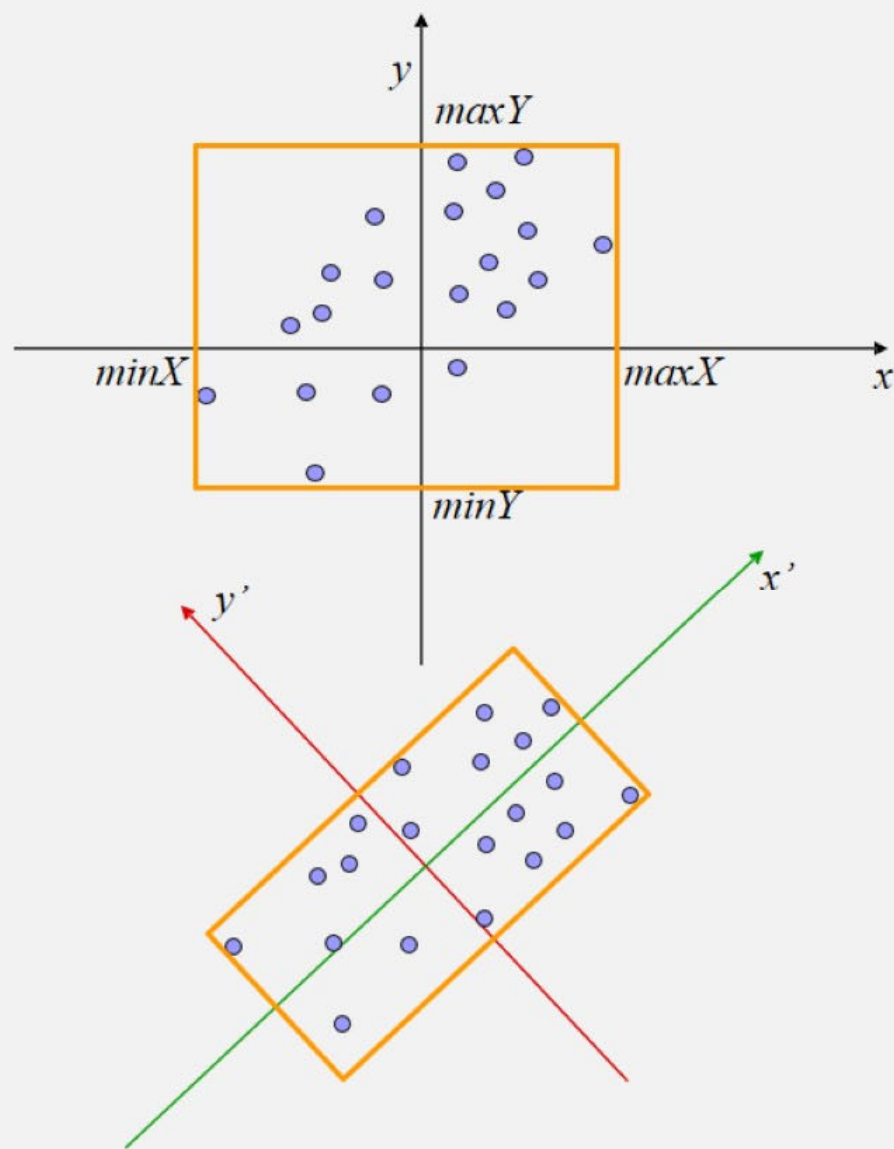
$$\min \sum \text{distances}^2$$

点云的基本特征和描述

➤ 点云的基本特征描述:

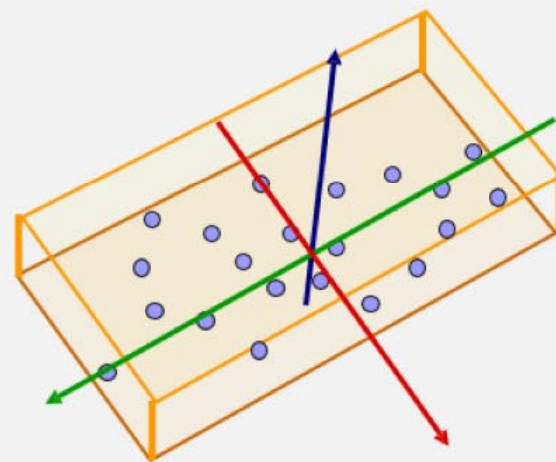
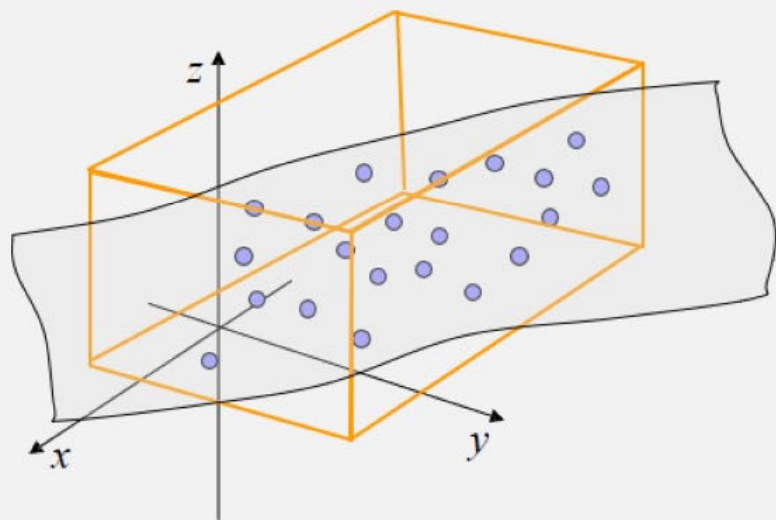


$$\min \sum distances^2$$



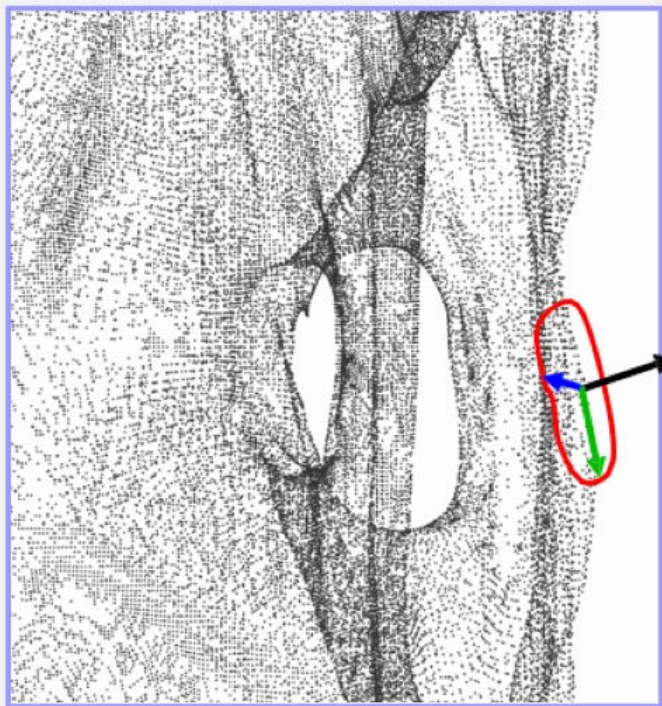
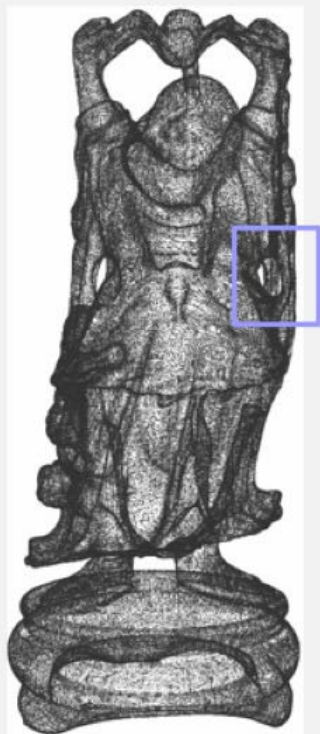
点云的基本特征和描述

➤ 点云的基本特征描述:

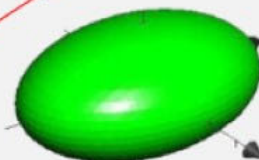
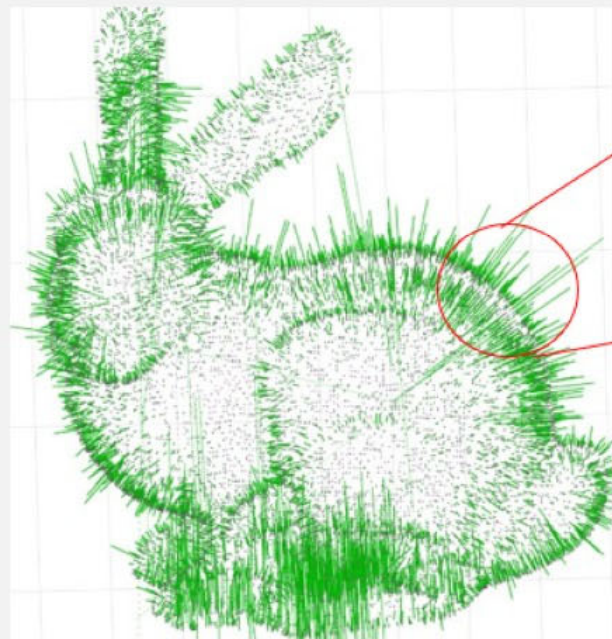


点云的基本特征和描述

➤ 点云的基本特征描述:

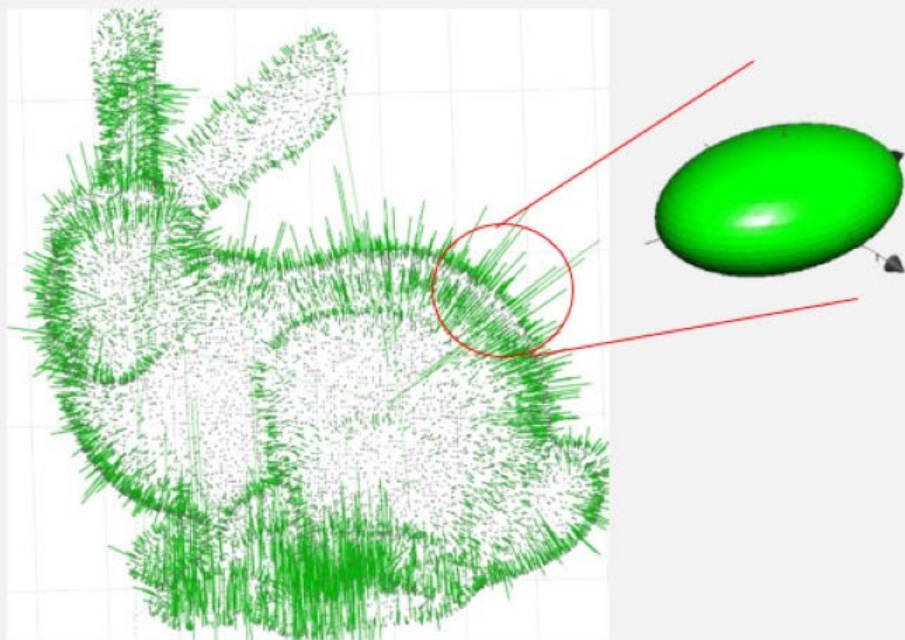


normal



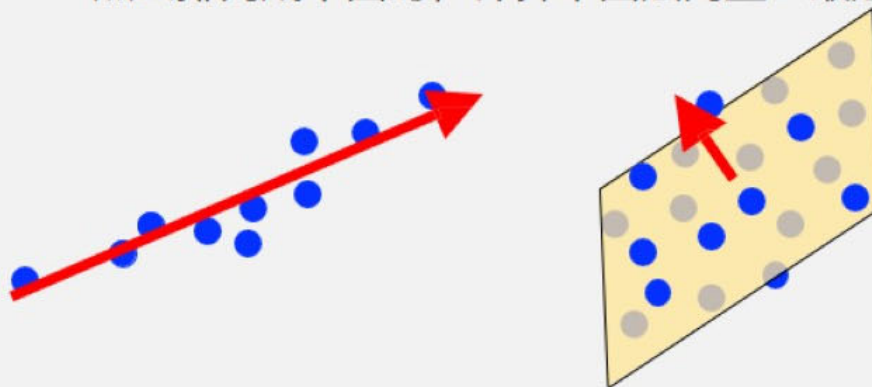
点云的基本特征和描述

➤ 点云的法向量估计:



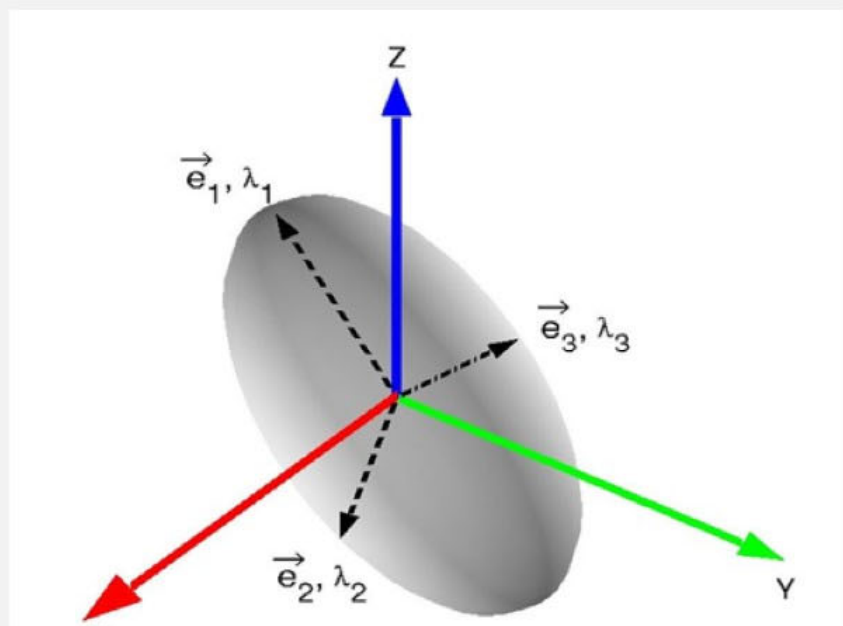
- PCA是指“主成分分析” Principle Components Analysis
- 使用的核心算法是矩阵的特征值分解

- 对于物体表面一点，将它和邻域的点云看成是空间椭球体
- 法向量方向对应椭球体的最短轴方向
- 基于矩阵特征值或者SVD分解求：
 - 法向量方向
 - (等效) 椭球体最短轴方向
 - 坐标协方差矩阵最小特征值对应的特征向量
- 排列成直线的点云方向获取：最长轴方向
- 点云排列成平面时，计算平面法向量：最短轴方向



点云的基本特征和描述

点云的协方差矩阵分解:



$$\Sigma = U\Lambda U^T$$

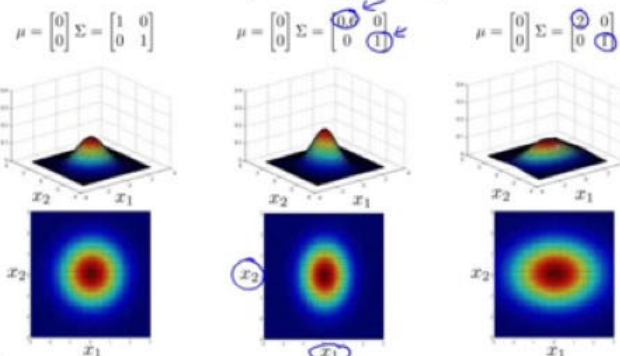
数据集在某个基上的投影值(也是在这个基上的坐标值)越分散, **方差越大**, 这个基保留的信息也就越多
信息量保存能力最大的基向量一定是**协方差矩阵的特征向量**, 并且这个特征向量保存的信息量就是它对应的特征值.

样本每一个特征的均值计算公式:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^N x_i$$

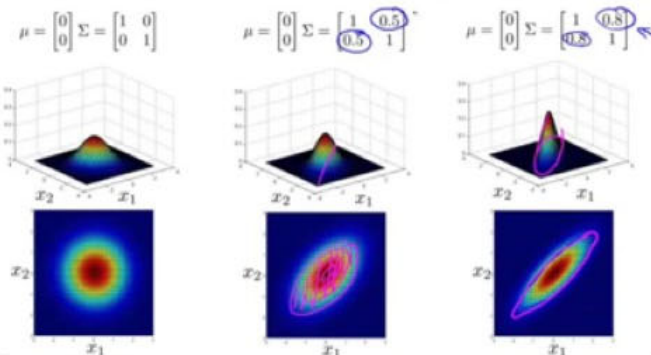
样本方差的计算公式:

Multivariate Gaussian (Normal) examples



样本X和Y的协方差:

Multivariate Gaussian (Normal) examples



设数据集为:

那么协方差矩阵为

$$C = \begin{pmatrix} \text{cov}(x,x) & \text{cov}(x,y) & \text{cov}(x,z) \\ \text{cov}(y,x) & \text{cov}(y,y) & \text{cov}(y,z) \\ \text{cov}(z,x) & \text{cov}(z,y) & \text{cov}(z,z) \end{pmatrix}$$

$$\frac{1}{n} X X^T$$

<https://blog.csdn.net/u014485485/article/details/78408886>

点云的基本特征和描述

➤ 点云的PCA:

- 定义

数据点云定义为 $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n \in R^d$

$$\mathbf{x}_1 = \begin{pmatrix} x_1^1 \\ x_1^2 \\ \vdots \\ x_1^d \end{pmatrix}, \quad \mathbf{x}_2 = \begin{pmatrix} x_2^1 \\ x_2^2 \\ \vdots \\ x_2^d \end{pmatrix}, \quad \dots, \quad \mathbf{x}_n = \begin{pmatrix} x_n^1 \\ x_n^2 \\ \vdots \\ x_n^d \end{pmatrix}$$

点云的基本特征和描述

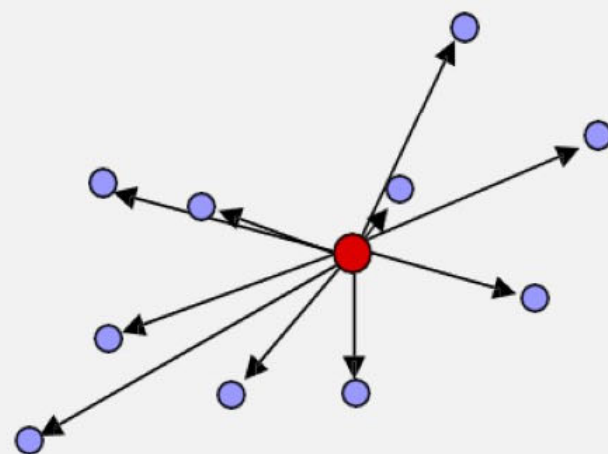
➤ 点云的PCA:

- 定义中心原点

中心原点应为点云的质心 $\mathbf{m}$ (Centroid), 满足以下条件:

$$\mathbf{m} = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i$$

$$\mathbf{m} = \operatorname{argmin}_{\mathbf{x}} \sum_{i=1}^n \|\mathbf{x}_i - \mathbf{x}\|^2$$



点云的基本特征和描述

➤ 点云的PCA:

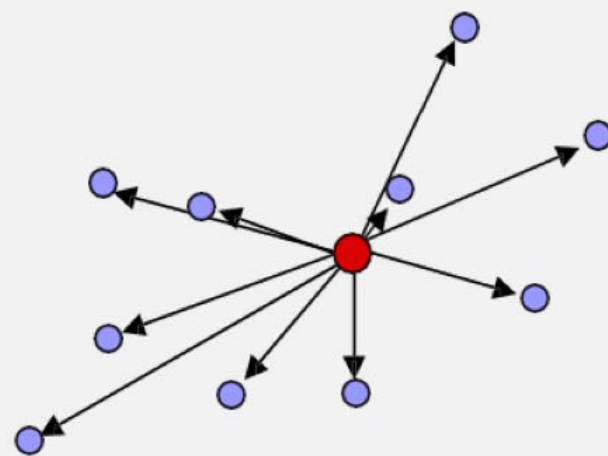
- 散布矩阵

定义: $\mathbf{y}_i = \mathbf{x}_i - \mathbf{m}, i = 1, 2, \dots, n$

$$S = YY^T$$

其中 Y 是 $d \times n$ 矩阵, $\mathbf{y}_k$ 为列向量 ($k = 1, 2, \dots, n$)

$$S = \underbrace{\begin{pmatrix} y_1^1 & y_2^1 & \cdots & y_n^1 \\ y_1^2 & y_2^2 & \cdots & y_n^2 \\ \vdots & \vdots & & \vdots \\ y_1^d & y_2^d & \cdots & y_n^d \end{pmatrix}}_Y \underbrace{\begin{pmatrix} y_1^1 & y_1^2 & \cdots & y_1^d \\ y_2^1 & y_2^2 & \cdots & y_2^d \\ \vdots & \vdots & & \vdots \\ y_n^1 & y_n^2 & \cdots & y_n^d \end{pmatrix}}_{Y^T}$$



点云的基本特征和描述

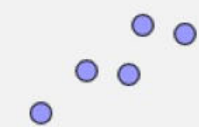
➤ 点云的PCA:

- 投影点的方差

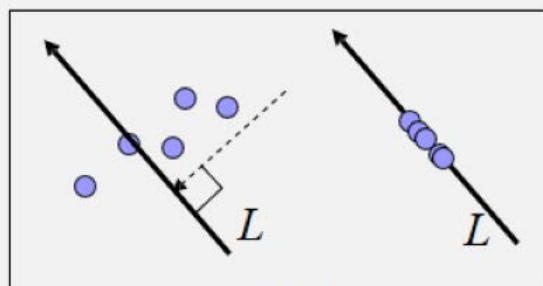
S 表示数据不同方向的方差

将点 $\mathbf{x}_i$ 投影到通过质心 $\mathbf{m}$ 的直线 L , 投影点 $\mathbf{x}'_i$ 的方差为:

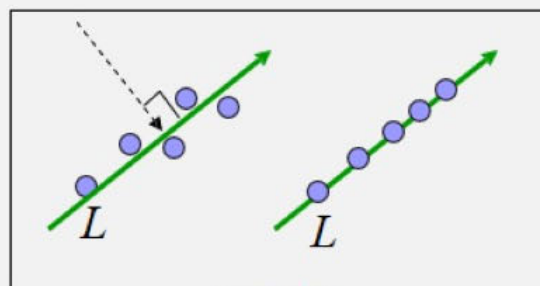
$$\text{var}(L) = \frac{1}{n} \sum_{i=1}^n \|\mathbf{x}'_i - \mathbf{m}\|^2$$



原始点云



小方差



大方差

点云的基本特征和描述

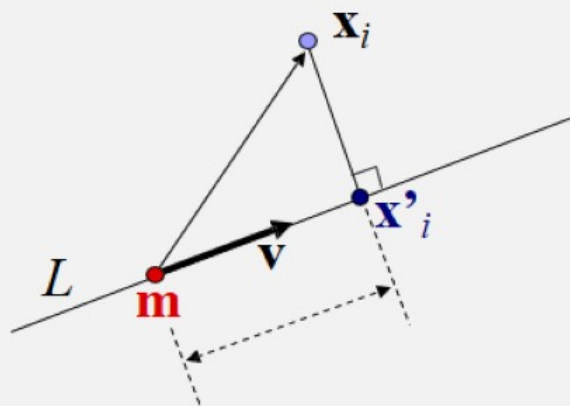
➤ 点云的PCA:

- 投影点的方差

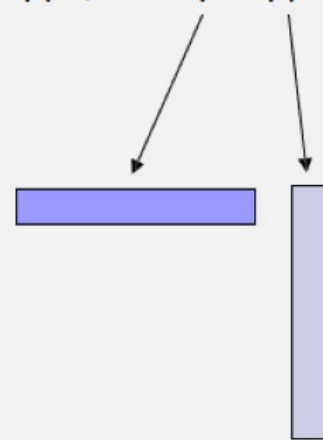
给定方向向量 $\mathbf{v}$, $\|\mathbf{v}\| = 1$,

点 $\mathbf{x}_i$ 在直线 $L = \mathbf{m} + \mathbf{v}t$ 上的投影为:

$$\|\mathbf{x}'_i - \mathbf{m}\| = \langle \mathbf{v}, \mathbf{x}_i - \mathbf{m} \rangle / \|\mathbf{v}\| = \langle \mathbf{v}, \mathbf{y}_i \rangle = \mathbf{v}^T \mathbf{y}_i$$



$$\text{内积: } \langle \mathbf{v}, \mathbf{w} \rangle = \mathbf{v}^T \mathbf{w} = \mathbf{w}^T \mathbf{v}$$

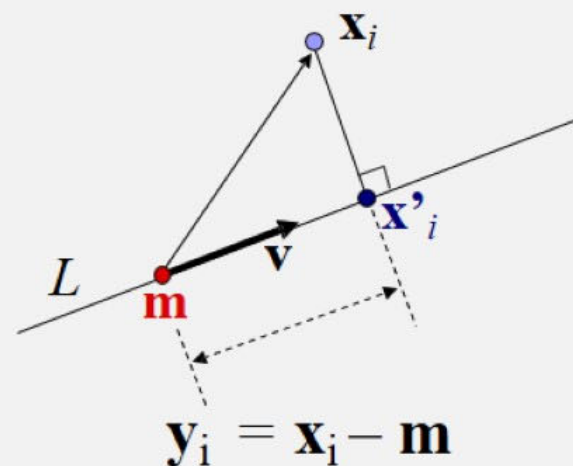


点云的基本特征和描述

点云的PCA:

- 投影点的方差
所以有:

$$\begin{aligned}\text{var}(L) &= \frac{1}{n} \sum_{i=1}^n \| \mathbf{x}'_i - \mathbf{m} \|^2 = \frac{1}{n} \sum_{i=1}^n (\mathbf{v}^T \mathbf{y}_i)^2 = \frac{1}{n} \| \mathbf{v}^T Y \|^2 = \\ &= \frac{1}{n} \| Y^T \mathbf{v} \|^2 = \frac{1}{n} \langle Y^T \mathbf{v}, Y^T \mathbf{v} \rangle = \frac{1}{n} \mathbf{v}^T Y Y^T \mathbf{v} = \frac{1}{n} \mathbf{v}^T S \mathbf{v} = \frac{1}{n} \langle S \mathbf{v}, \mathbf{v} \rangle\end{aligned}$$



$$\sum_{i=1}^n (\mathbf{v}^T \mathbf{y}_i)^2 = \sum_{i=1}^n \left((v^1 \ v^2 \ \dots \ v^d) \begin{pmatrix} y_i^1 \\ y_i^2 \\ \vdots \\ y_i^d \end{pmatrix} \right)^2 = \left\| (v^1 \ v^2 \ \dots \ v^d) \begin{pmatrix} y_1^1 & y_1^2 & \dots & y_1^d \\ y_2^1 & y_2^2 & \dots & y_2^d \\ \vdots & \vdots & \dots & \vdots \\ y_n^1 & y_n^2 & \dots & y_n^d \end{pmatrix} \right\|^2 = \| \mathbf{v}^T Y \|^2$$

点云的基本特征和描述

➤ 点云的PCA:

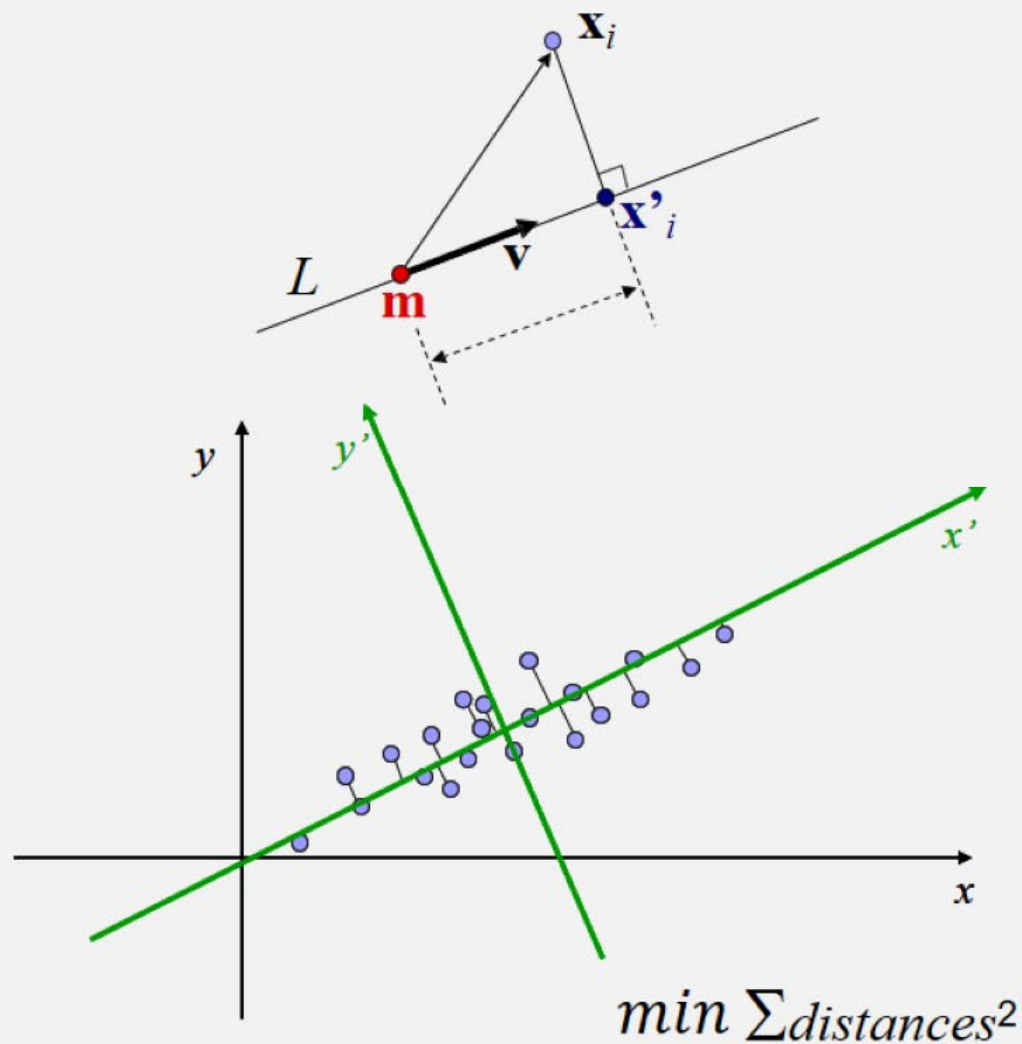
- 投影点的方差
所以现有: $\text{var}(L) = \langle S\mathbf{v}, \mathbf{v} \rangle$
- 定理:

$$f: \{\mathbf{v} \in R^d \mid \|\mathbf{v}\| = 1\} \rightarrow R,$$

$$f(\mathbf{v}) = \langle S\mathbf{v}, \mathbf{v} \rangle \quad (S \text{ 为对称矩阵}).$$

则在 S 的特征向量处求出 f 的极值

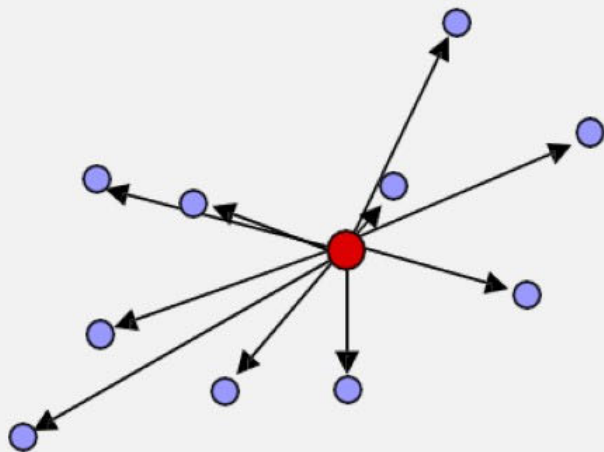
所以, S 的特征向量是最大方差的方向



点云的基本特征和描述

➤ 点云的PCA:

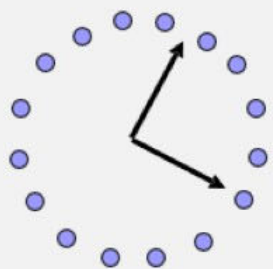
- 首先将数据去中心化, 得到去中心化后的 $\mathbf{y}_1, \mathbf{y}_2, \dots, \mathbf{y}_n \in R^d$
- 然后计算协方差矩阵 (散布矩阵) $S = YY^T$
- S 的特征向量是最大方差的方向
- 用特征值分解方法求协方差矩阵的特征值与特征向量
- 对特征值从大到小排序, 选择其中最大的k个 $\lambda_1 \geq \lambda_2 \geq \lambda_3 \geq 0$
- 然后将其对应的k个特征向量分别作为列向量组成特征向量矩阵的正交基



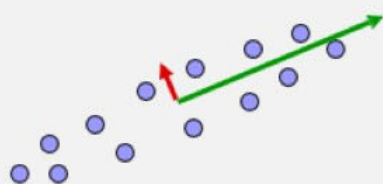
$$S = \begin{pmatrix} | & | & | & | \\ \mathbf{v}_1 & \mathbf{v}_2 & & \mathbf{v}_d \\ | & | & | & | \end{pmatrix} \begin{pmatrix} \lambda_1 & & & \\ & \lambda_2 & & \\ & & \ddots & \\ & & & \lambda_d \end{pmatrix} \begin{pmatrix} \hline \mathbf{v}_1 \\ \hline \mathbf{v}_2 \\ \hline \\ \hline \mathbf{v}_d \\ \hline \end{pmatrix}$$

点云的基本特征和描述

➤ 点云的特征值特征向量:

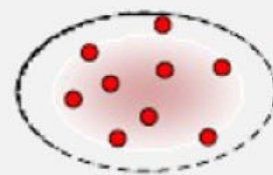
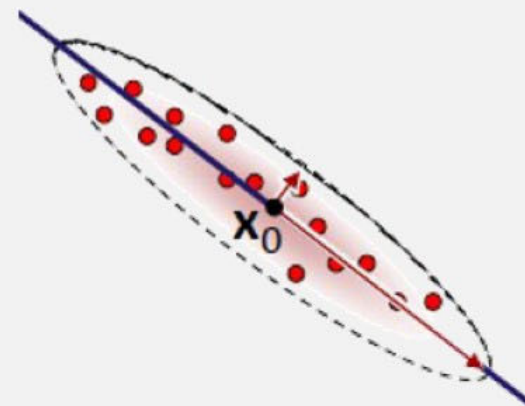


$$V \begin{pmatrix} \lambda & \\ & \lambda \end{pmatrix} V^T$$

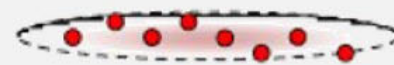


$$V \begin{pmatrix} \lambda & \\ & \mu \end{pmatrix} V^T$$

$\lambda \gg \mu$



(λ_2 / λ_1) larger



(λ_2 / λ_1) small

点云的基本特征和描述

➤ 点云的特征值特征向量: $\lambda_1 \geq \lambda_2 \geq \lambda_3 \geq 0$ $e_i = \lambda_i / \sum \lambda_i$ $i \in \{1, 2, 3\}$

特征	表达式	特征	表达式
线性	$L_\lambda = \frac{\lambda_1 - \lambda_2}{\lambda_1} = \frac{e_1 - e_2}{e_1}$	各向异性	$L_\lambda = \frac{\lambda_1 - \lambda_3}{\lambda_1} = \frac{e_1 - e_3}{e_1}$
平面性	$P_\lambda = \frac{\lambda_2 - \lambda_3}{\lambda_1} = \frac{e_2 - e_3}{e_1}$	特征熵	$E_\lambda = -\sum_{i=1}^3 e_i \ln(e_i)$
散度性(球度性)	$S_\lambda = \frac{\lambda_3}{\lambda_1} = \frac{e_3}{e_1}$	迹	$T_\lambda = \frac{2}{\pi} \arctan(\lambda_1 + \lambda_2 + \lambda_3)$
全方差	$O_\lambda = 3\sqrt[3]{e_1 e_2 e_3}$	曲率变化	$C_\lambda = \frac{3\lambda_3}{\lambda_1 + \lambda_2 + \lambda_3} = 3e_3$